MPC23.jpg

SECTION A

Question 1

1(a) How 8086 can access a 16-bit word under odd/even address? (4%)

The 8086 has a **16-bit data bus** split into two 8-bit banks:

- **Even bank (low byte)** – connected to data lines D0–D7
- **Odd bank (high byte)** – connected to data lines D8–D15

The processor uses **Bus High Enable (BHE)** and **A0** signals to decide which bank is active.

(i) Low byte at an odd address (e.g., 1001H)

- **Problem:** 16-bit word spans from odd → next even address (1001H and 1002H)
- **Solution:** 8086 needs **two bus cycles**:
  - Cycle 1: Read low byte from odd address (1001H) → D0–D7
  - Cycle 2: Read high byte from next even address (1002H) → D8–D15

(ii) Low byte at an even address (e.g., 1000H)

- **Solution:** One bus cycle (both bytes fetched together)
  - Even address 1000H → low byte on D0–D7
  - Next odd address 1001H → high byte on D8–D15

Diagram:

text

```
8086 Memory Access - Even Address (1000H)
┌──────────────┐      ONE BUS CYCLE      ┌──────────────────┐
│   8086 CPU   │──────────────────────→│ Even: 1000H (Low)│
│              │                         │ Odd:  1001H (High)│
└──────────────┘                         └──────────────────┘

8086 Memory Access - Odd Address (1001H)
┌──────────────┐      BUS CYCLE 1        ┌──────────────────┐
│   8086 CPU   │──────────────────────→│ Odd:  1001H (Low)│
│              │      BUS CYCLE 2        │ Even: 1002H (High)│
└──────────────┘──────────────────────→└──────────────────┘
```

ELI5:
Imagine you have two hands (low byte bank and high byte bank). If a 2-byte word starts at an even house number, you can knock both doors at once. If it starts at an odd number, you must knock one door, then walk to the next house and knock again.

1(b) Different types of core registers of Cortex M3 processor (2%)

Register Group	Registers	Purpose
Low Registers	R0–R7	General purpose, 32-bit, all instructions can access
High Registers	R8–R12	General purpose, but limited instructions
Stack Pointer	SP (R13)	Two: MSP (main) and PSP (process)
Link Register	LR (R14)	Stores return address after subroutine call
Program Counter	PC (R15)	Points to next instruction
Program Status Register	xPSR	Contains flags (Zero, Carry, Overflow, etc.)
Control Register	CONTROL	Selects stack pointer mode

Real-world example:
Like a carpenter's tool belt:

- R0–R12 = screwdrivers, hammers (general tools)

- SP = where you keep your current blueprint (stack)
- LR = sticky note saying “come back here after finishing task”
- PC = your finger pointing at next step in instruction manual

1(c) Physical address from 0A32:0028H (1%)

Formula: Physical = Segment × 10H + Offset
= 0A32H × 10H + 0028H
= 0A320H + 0028H
= **0A348H**

ELI5:
Think of segment as “street number × 16” and offset as “house number”.
Street 0A32 × 16 = 0A320, add house 0028 → 0A348.

Question 2

2(a) Explain instructions with diagram (4%)

Assume initial **R0 = 0b 1100 1011** (8-bit example for clarity)

(i) ASR #2 (Arithmetic Shift Right by 2)

- Preserves sign bit (MSB)
- Shift right, fill with sign bit

text

```
Before: 1 1 0 0 1 0 1 1  (MSB=1, negative)
After:  1 1 1 1 0 0 1 0  (shifted, sign extended)
```

Diagram:

text

```
MSB→|1|1|0|0|1|0|1|1| ←LSB
     | ↓  ↓              (shift right 2)
     | 1|1|1|1|0|0|1|0|
        ↑  ↑ sign bit fills
```

(ii) LSR #4 (Logical Shift Right by 4)

- Shift right, fill with zeros
- LSB bits lost

text

```
Before: 1 1 0 0 1 0 1 1
After:  0 0 0 0 1 1 0 0
```

(iii) LSL #1 (Logical Shift Left by 1)

- Shift left, fill LSB with zero

text

```
Before: 1 1 0 0 1 0 1 1
After:  1 0 0 1 0 1 1 0
```

(iv) ROR #4 (Rotate Right by 4)

- Bits shifted out from LSB come back into MSB

text

```
Before: 1 1 0 0 1 0 1 1
After:  1 0 1 1 1 1 0 0
(Last 4 bits 1011 move to front)
```

ELI5:

- **ASR:** Like sliding a deck of cards right, but keep the top card’s suit (sign)
- **LSR:** Slide right, push zeros from left
- **LSL:** Slide left, push zeros from right
- **ROR:** Like a rotating conveyor belt – what falls off right end comes back left end

2(b) Clock sources for STM32F103x (4%)

Clock Source	Type	Frequency	Use
HSI	Internal RC	8 MHz	Default, quick startup

Clock Source	Type	Frequency	Use
HSE	External crystal	4–16 MHz	High accuracy
PLL	Phase Locked Loop	Up to 72 MHz	Multiply HSI/HSE for max speed
LSI	Low-speed internal	40 kHz	Watchdog, RTC
LSE	Low-speed external	32.768 kHz	Accurate RTC

Default clock source: HSI (8 MHz)

ELI5:

- HSI = cheap wall clock (good enough, always there)
- HSE = atomic clock (expensive but accurate)
- PLL = coffee machine that turns 1 coffee into 9 shots (speed boost)
- LSI/LSE = wristwatch battery (low power, keeps time while sleeping)

Question 3

3(a) Explain STM32 code with diagram (4%)

assembly

```
CMP R0, R1;          // Compare R0 with R1 → sets flags (Z, N,
C, V)
ITT GT;              // If-Then block for 2 instructions,
condition GT (Greater Than)
CMPGT R2, R3;        // Execute if R0 > R1: compare R2 and R3
MOVGT R4, R5;        // Execute if R0 > R1: move R5 into R4
```

Diagram:

text

```
        Start
          |
          ▼
    Compare R0, R1
          |
      ┌───┴───┐
      │ R0 > R1?│
      └───┬───┘
          |
     No   |  Yes
          |    ▼
    Skip  |  Compare R2,R3
    both  |    |
          |    ▼
          |  Move R5→R4
          |    |
          ▼    ▼
        Continue
```

Real-world example:
If (score > passing_marks) {
compare(answer, correct);
copy_grade_to_report;
}

ELI5:
`ITT GT` = "If the first comparison says R0 is greater, then do the next two instructions. Otherwise, skip them like they don't exist."

3(b) MOVT instruction with example (2%)

MOVT = **Move Top** – loads the **upper 16 bits** of a 32-bit register without changing lower 16 bits.

Example:

assembly

```
MOV  R0, #0x1234    ; R0 = 0x00001234
MOVT R0, #0xABCD    ; R0 = 0xABCD1234
```

Real-world use:
Loading a 32-bit address in Thumb-2 code (ARM instructions are only 16/32 bits wide, can't load full 32-bit constant in one go).

ELI5:
Imagine a 4-digit display:

- `MOV` sets the right two digits
- `MOVT` sets the left two digits without touching right digits

3(c) Explain GPIOC_CRH code (2%)

c

```
GPIOC_CRH &= ~(0xF << 20);
```

Breakdown:

- `GPIOC_CRH` = Control Register High for GPIOC port (pins 8–15)
- `0xF << 20` = binary `1111` shifted left 20 bits → affects bits 20–23
- `~` = bitwise NOT → `1111` becomes `0000` in those positions
- `&=` = AND-assign → clears bits 20–23 to 0, leaves others unchanged

Effect: Configures **GPIO pin PC13** (since each pin uses 4 bits in CRH) as **input floating/reset state**.

ELI5:
You have a row of 32 light switches. `0xF << 20` points to switches 20,21,22,23. `~` flips them off. `&=` says "turn off only those switches, leave others as they were."

Question 4

4(a) Bit-band alias address calculation (3%)

Given: Peripheral region bit-band. Bit 6 of RCC_APB2ENR at address 0x4002 1018.

Formula for alias address:

text

```
Alias = 0x42000000 + (byte_offset × 32) + (bit_number × 4)
```

Where: `byte_offset = target_address - 0x40000000`

Step 1: byte_offset = 0x40021018 − 0x40000000 = 0x21018

Step 2: Alias = 0x42000000 + (0x21018 × 32) + (6 × 4)

Step 3: 0x21018 × 32 = 0x21018 × 0x20 = 0x420300 (Calculate: 0x21018 << 5)

Step 4: 6 × 4 = 24 = 0x18

Step 5: 0x42000000 + 0x420300 = 0x42420300
Add 0x18 → **0x42420318**

Answer: 0x42420318

ELI5:
Bit-banding = giving each single bit its own full 32-bit mailbox. Instead of opening the whole byte, you can write directly to one bit's private address.

4(b) STRD instruction execution (2%)

`STRD R3, [R4]` stores **two 32-bit values** (R3 and R4) into memory at address in R4.

Given R4 = 0x43000000

- **STRD stores R3 to [R4] and R4 to [R4+4]**

Case	R3 value	Memory at 0x43000000	Memory at 0x43000004
(i)	0xFF	0x000000FF	0x43000000
(ii)	0x21	0x00000021	0x43000000
(iii)	0x00	0x00000000	0x43000000
(iv)	0x10	0x00000010	0x43000000

Important: R4's original value is also stored as the second word.

ELI5:
STRD = "Store Double" – like putting two big suitcases (R3 and R4) into two consecutive lockers starting at address R4.

4(c) Key functions of RCC_APB2ENR register (2%)

RCC_APB2ENR = Reset and Clock Control – APB2 Peripheral Clock Enable Register

Key functions:

1. **Enable/disable clocks** for high-speed peripherals on APB2 bus (GPIOs, ADC, SPI1, USART1, EXTI, AFIO)
2. **Reduce power consumption** by turning off unused peripherals
3. **Initialize peripherals** – must enable clock before accessing peripheral registers

Bit	Peripheral	Typical use
2	GPIOAEN	Enable GPIO Port A
3	GPIOBEN	Enable GPIO Port B
4	GPIOCEN	Enable GPIO Port C
9	ADC1EN	Enable ADC1
14	USART1EN	Enable USART1

ELI5:
RCC_APB2ENR is like a **master power strip** for your lab equipment. Before using an oscilloscope (GPIO), you must turn on its outlet. After finishing, turn it off to save electricity (battery).

SECTION B

Question 5

5(a) Career paths and common courses: CS vs CE (3%)

Career Path Differences:

Aspect	Computer Science (CS)	Computer Engineering (CE)
Focus	Software, algorithms, data, theory	Hardware-software interface, embedded systems
Typical jobs	Software dev, AI/ML engineer, web dev, data scientist	Embedded engineer, FPGA designer, robotics, IoT
Industries	Tech companies, finance, startups	Semiconductor, automotive, aerospace, consumer electronics
Example role	Building an app like Uber	Programming ABS brake controller in a car

Common undergraduate courses:

1. Programming Fundamentals (C, Python)
2. Data Structures & Algorithms
3. Digital Logic Design
4. Computer Architecture
5. Operating Systems
6. Microprocessors & Microcontrollers (this course!)
7. Discrete Mathematics

ELI5:

- **CS** = Chef who creates recipes (software)
- **CE** = Chef who also builds and fixes the oven (hardware + software)
- **Common courses** = Both learn knife skills, food safety, and menu planning

5(b) Purpose of HAL in STM32 development (2%)

HAL = Hardware Abstraction Layer

Purposes:

1. **Portability** – Write code once, run on any STM32 family with minimal changes
2. **Simplification** – High-level functions like `HAL_GPIO_WritePin()` instead of bit manipulation
3. **Speed up development** – No need to read 1000-page reference manuals for basic tasks

4. **Standardized API** – Same function names across all STM32 MCUs

Real-world example:
Without HAL: `*(volatile uint32_t*)0x40010C10 |= (1<<13);` (turn on LED)
With HAL: `HAL_GPIO_WritePin(LED_PORT, LED_PIN, GPIO_PIN_SET);`

ELI5:
HAL is like a **universal TV remote** – same buttons (power, volume, channel) work on Sony, Samsung, LG. You don't need to know each TV's secret codes.

5(c) Does 8086 follow Harvard architecture? Explain (3%)

Answer: NO. 8086 follows **Von Neumann architecture**.

Explanation:

Feature	Von Neumann (8086)	Harvard
Memory for code & data	Single shared memory	Separate memories
Bus for code & data	Single bus	Two buses (or more)
Can fetch code & data simultaneously?	No	Yes

8086 specific:

- Code and data reside in the **same memory space** (1MB)
- Same bus carries both instructions and data
- Can't fetch next instruction while reading/writing data

Proof:
Segment registers (CS, DS, SS, ES) all point into the same physical memory. The processor fetches code from CS:IP, then next cycle reads data from DS:SI – but using the **same** bus, causing wait states.

Real-world effect:
8086 at 5 MHz is slower than a Harvard-based 8051 at 12 MHz for some tasks, because 8086 wastes cycles switching between code fetch and data access.

ELI5:

- **Von Neumann (8086)** = One waiter carries both your food order (code) and your drink (data) using the same hallway.
- **Harvard** = Two waiters: one only for food orders, one only for drinks – they can work simultaneously.

Final ELI5 of all major terms

Term	ELI5
ASR	Sliding cards right, keeping the top card's face up
LSR	Sliding cards right, pushing in blank cards from left
ROR	Conveyor belt – bits that fall off come back around
Bit-banding	Giving every single Lego brick its own mailbox
MOVT	Filling left two digits of a 4-digit display
ITT GT	"If the score is higher, then do these two tasks, otherwise skip"
HAL	Universal remote for any STM32 TV
Von Neumann	One hallway for both people and packages
Harvard	Separate hallway for people, separate for packages

Good luck tomorrow! Focus on **diagrams** (they give extra marks), **formulas** (physical address, bit-band), and **real-world examples** (make the examiner feel you understand, not memorize). You've got this! 🚀

MPC23_2.jpg

JPG 3.64MB

Here is the complete solution for **Section B (Questions 6, 7, 8)** from your second image.
I have followed exam-oriented style: **diagrams, real-world examples, tables, and ELI5 explanations** for each term.

SECTION B (Continued)

Question 6

6(a) Advantages and Disadvantages of RISC vs CISC Processor

Feature	RISC (e.g., ARM, Cortex-M)	CISC (e.g., x86, 8086)
Full form	Reduced Instruction Set Computer	Complex Instruction Set Computer
Instruction size	Fixed (32-bit)	Variable (1–15 bytes)
Number of instructions	Few (~50–100)	Many (~200–500)
Addressing modes	Few (3–5)	Many (10–20)
Execution time	One instruction per clock cycle	Multiple clock cycles per instruction

Advantages & Disadvantages Table

	RISC	CISC
Advantages	1. Faster clock speeds 2. Simpler hardware (cheaper) 3. Pipelining is easy 4. Low power consumption	1. Smaller code size 2. Less RAM needed 3. Complex tasks in one instruction 4. Easier for compilers (historically)
Disadvantages	1. More instructions to do complex tasks 2. Larger program memory needed 3. Harder to write assembly manually	1. Slower clock speeds 2. More power hungry 3. Complex hardware (expensive) 4. Hard to pipeline

Real-world example:

- **RISC** = Swiss army knife with 5 tools (simple, fast, efficient)
- **CISC** = Mechanic's giant tool chest (has a tool for everything, but heavy and slow to open)

ELI5:
RISC = bicycle (simple, fast, but you pedal each step)
CISC = motorcycle (complex engine, but one twist does many things)

6(b) Interface: 8 Switches → 8 LEDs on 8085 (each switch controls corresponding LED)

Requirement:
Switch #1 → LED #1, Switch #2 → LED #2, etc.
When switch is ON → LED ON. When switch OFF → LED OFF.

Hardware Design

text

```
            ADDRESS BUS (A0-A7)
                    │
8085 CPU ───────────┼──────────────────────────────┐
     │              │                              │
     │      A8-A15 (unused, tied to GND)           │
     │                                             │
     │     ┌──────────────┐                        │
     │     │   74LS138    │   Decoder              │
     │     │   (3:8)      │                        │
     │     │              │                        │
  IO/M ────┤G1            │   Y0 ──┐               │
     │     │              │        ▼               │
  A0 ────┤A               │      ┌──────┐          │
  A1 ────┤B               │      │ 74LS245│        │
  A2 ────┤C               │      │ Buffer │        │
     │     └──────────────┘      └──┬───┘          │
     │                              │              │
     │         DATA BUS (D0-D7)     │              │
     │          ┌───────────────────┘              │
     │          │                                  │
     ▼          ▼                                  ▼
┌──────────────────────┐              ┌──────────────┐
│  8-bit Input Port    │              │ 8-bit Output │
│  (e.g., 74LS244)     │              │ Port (e.g.,  │
│                      │              │   74LS373)   │
│   D0────Switch1      │              │   Q0────LED1 │
│   D1────Switch2      │              │   Q1────LED2 │
│   ...                │              │   ...        │
│   D7────Switch8      │              │   Q7────LED8 │
└──────────┬───────────┘              └──────┬───────┘
           │                                 │
           └──────────── RD ─────────────────┘
                      WR ────────────────────┘
```

Working Principle

1. **Read switches** from input port using `IN` instruction (when decoder selects input port)
2. **Write same data** to output port using `OUT` instruction
3. Repeat in infinite loop

8085 Assembly Code

assembly

```
START:  IN 80H        ; Read switches from input port address
80H
        OUT 81H       ; Write same value to output port (LEDs)
        JMP START     ; Loop forever
```

Address decoding:

- Input port selected when IO/M=1, A2–A0 = 000 → Y0 low → enables input buffer
- Output port selected when A2–A0 = 001 → Y1 low → enables output latch

ELI5:
Think of a mirror: whatever the switches show (fingers up/down), the LEDs copy exactly. The CPU is just a messenger reading left hand (switches) and copying to right hand (LEDs) over and over.

Question 7

7(a) Advantages and Disadvantages of Partial Address Decoding

Partial address decoding = using only **some** address lines to select a memory chip, ignoring others.
This causes **multiple addresses** to map to the same chip (address aliasing).

Diagram: Full vs Partial Decoding

text

```
Full Decoding (A0-A15 all used)          Partial Decoding
(only A13-A15 used)

┌──────────────────┐                     ┌──────────────────┐
│ 16 address lines │                     │ 16 address lines │
│ A0 A1 ... A15    │                     │ A0 A1 ... A15    │
│  ↓  ↓       ↓    │                     │  ↓  ↓       ↓    │
│ All lines        │                     │ Only A13,A14,A15 │
│ go to decoder    │                     │ go to decoder    │
│                  │                     │ A0-A12 ignored   │
│ Chip has ONE     │                     │ Chip has MANY    │
│ unique address   │                     │ alias addresses  │
└──────────────────┘                     └──────────────────┘
```

Advantages

Advantage	Explanation
Saves hardware	Fewer logic gates (no need for full decoder)
Simpler wiring	Less PCB traces
Cheaper	Can use simple AND/OR gates instead of 3:8 or 4:16 decoder
Faster decoding	Less propagation delay

Disadvantages

Disadvantage	Explanation
Address aliasing	Same chip appears at multiple addresses (wastes address space)
Memory conflicts	Two chips might respond to same address if not careful
Difficult to expand	Adding more memory requires redesigning decoding
Debugging headache	Programmer might write to address X but chip at Y responds

Example of Partial Decoding

Scenario: 8088 system with 8KB EPROM (2764). Use only A13, A14, A15 to select chip.

text

```
A15 A14 A13 → Chip Select
  0   0   0 → EPROM enabled
  0   0   1 → EPROM enabled (alias!)
  0   1   0 → EPROM enabled (alias!)
  ... etc.
```

Result: Same 8KB EPROM appears at **8 different 8KB blocks** (total 64KB aliased).

ELI5:
Full decoding = each house has a unique street number.
Partial decoding = whole block shares the same number – mail goes to one house, but other houses also get copies of same mail.

7(b) Address range for each memory chip (2764 EPROM)

Given:

- A13, A14, A15 → select the chip (decoder inputs)
- A16, A17, A18, A19 → enable the decoder
- 2764 = 8K × 8 EPROM → needs 13 address lines (A0–A12)
- Total address bus = 20 bits (A0–A19) in 8086/8088 minimum mode

Step-by-step calculation

1. Chip capacity: 8KB = 2^{13} bytes → uses A0 to A12

2. Decoder inputs: A13, A14, A15 (3 lines → 8 possible combinations)

3. Decoder enable: A16, A17, A18, A19 must be fixed value to enable decoder

Assume decoder enable when A19 A18 A17 A16 = 0000 (common choice)

Address range formula

```
Base address = (Enable bits) × 2^16 + (Decoder select bits) × 2^13
```

Decoder truth table:

A19..A16	A15	A14	A13	Selected chip	Start Address	End Address
0000	0	0	0	Chip 0	00000H	01FFFH
0000	0	0	1	Chip 1	02000H	03FFFH
0000	0	1	0	Chip 2	04000H	05FFFH
0000	0	1	1	Chip 3	06000H	07FFFH
0000	1	0	0	Chip 4	08000H	09FFFH
0000	1	0	1	Chip 5	0A000H	0BFFFH

A19..A16	A15	A14	A13	Selected chip	Start Address	End Address
0000	1	1	0	Chip 6	0C000H	0DFFFH
0000	1	1	1	Chip 7	0E000H	0FFFFH

Address range for one chip (e.g., when A15 A14 A13 = 000)

- Start: A19..A16=0000, A15=0, A14=0, A13=0, A12..A0=000...0
  = `0000 0 00 0 0000000000000` binary = **00000H**
- End: A12..A0 = all 1s = 1FFFH
  = `0000 0 00 1 1111111111111` = **01FFFH**

Therefore each 2764 occupies 8KB space, 8KB apart.

ELI5:
Think of A16–A19 as the "building number" (fixed for decoder), A13–A15 as "floor number" (which chip), and A0–A12 as "room number" (inside the chip).

Question 8

8(a) Interface: 8 LEDs connected to 8085 – Moving light-spot effect

Requirement: LEDs turn on in sequence like a light spot moving left to right, then right to left (or loop).

Hardware Design (simpler than Q6b)

text

```
                ADDRESS BUS (A0-A7)
                        │
    8085 CPU ───────────┼──────────────────────────┐
         │              │                          │
         │      A8-A15 (unused)                    │
         │                                         │
         │     ┌──────────────┐                    │
         │     │   74LS138    │                    │
         │     │   (3:8)      │                    │
       IO/M ───┤G1            │                    │
       A0   ───┤A             │ Y0 ────────────┐   │
       A1   ───┤B             │                ▼   │
       A2   ───┤C             │              ┌─────┐
         │     └──────────────┘              │74LS373│
         │                                   │Latch  │
         │ DATA BUS (D0-D7)                  │       │
         └───────────────────────────────────│D0..D7 │
                                             │  Q0   │──LED1
                                             │  Q1   │──LED2
                                             │  ...  │
                                             │  Q7   │──LED8
                                             └─────┘
```

Assembly Code (Moving light spot left to right, then loop)

assembly

```
START:  MVI A, 80H      ; Start with leftmost LED (1000 0000)
        MVI B, 08H      ; Counter for 8 LEDs

LOOP:   OUT 81H         ; Send pattern to LEDs
        CALL DELAY      ; Wait so movement is visible
        RRC             ; Rotate right (move spot to next LED)
        DCR B           ; Decrement counter
        JNZ LOOP        ; Repeat 8 times
        JMP START       ; Restart from beginning
```

For bidirectional (back and forth):

assembly

```
        MVI A, 80H      ; Start LED7 ON
FORWARD: OUT 81H
        CALL DELAY
        RRC
        JNC FORWARD     ; Continue until carry (after 8
rotations)

BACKWARD:MVI A, 01H     ; Start LED0 ON
BACK:   OUT 81H
        CALL DELAY
        RLC
        JNC BACK
        JMP FORWARD
```

ELI5:
Imagine a single flashlight moving across 8 buckets. `RRC` shifts the light one bucket right each time. `DELAY` controls speed so human eye sees movement, not just all LEDs on at once.

8(b) Compare Memory Mapped I/O and Isolated I/O

Feature	Memory Mapped I/O	Isolated I/O (Port I/O)
Used in	ARM, Cortex-M, 68000	x86, 8085, 8086
Separate instructions?	No (same MOV/LDR)	Yes (IN, OUT)
Address space	Shared with memory	Separate (64KB in 8085)
Control signal	MEMR, MEMW	IOR, IOW (or IO/M)
Max I/O devices	Limited by memory space	Limited by I/O space
Decoding complexity	Simpler (use memory decoder)	Needs separate I/O decoder
Arithmetic on I/O?	Yes (ADD, SUB directly)	No (must read, then ALU)
Example instruction	`MOV [2000H], AL`	`OUT 80H, AL`

Diagram

text

```
Memory Mapped I/O                    Isolated I/O
┌─────────────────┐                  ┌─────────────────┐
│   64KB Memory   │                  │   64KB Memory   │
│   Space         │                  │   Space         │
├─────────────────┤                  ├─────────────────┤
│ RAM             │                  │ RAM             │
│ ROM             │                  │ ROM             │
│ I/O Device at   │                  └─────────────────┘
│ 0x2000 (uses    │
│ same addresses  │                  ┌─────────────────┐
│ as memory)      │                  │   64KB I/O      │
└─────────────────┘                  │   Space         │
                                     │ (separate bus)  │
                                     ├─────────────────┤
                                     │ Port 80H        │
                                     │ Port 81H        │
                                     │ ...             │
                                     └─────────────────┘
```

Advantages/Disadvantages

Aspect	Memory Mapped	Isolated I/O
Pros	Rich instruction set, no special I/O instructions, faster for complex operations	No memory conflict, simpler I/O devices, safe from stray memory writes
Cons	Reduces memory available, can cause bus contention	Limited instructions, slower, needs IN/OUT

ELI5:

- **Memory mapped** = Your desk drawer holds both books and snacks (shared space) – convenient but less space for books.
- **Isolated I/O** = Snacks in the fridge, books on shelf – separate spaces, but need two keys (IN and OUT instructions).

Final Summary Table for Quick Revision

Term	ELI5
RISC	Bicycle – simple, fast, efficient
CISC	Motorcycle – powerful, complex, heavy

Term	ELI5
Partial decoding	Whole street shares same house number
Full decoding	Every house has unique number
Memory mapped I/O	Office desk has both work and coffee
Isolated I/O	Coffee in kitchen, work at desk
Moving light spot	Flashlight moving across buckets
2764 EPROM	8K page of sticky notes (write once with UV eraser)

Good luck tomorrow! Draw diagrams for interface questions – examiners love them. Practice writing `IN`, `OUT`, `RRC`, `RLC` instructions. You will do great! 💡

MPC22_1.jpg

JPG 3.29MB

Here is the complete, exam-oriented solution for **Section A (Questions 1–4)** from your MPC22_1.jpg paper.
I have included **diagrams, tables, real-world examples, and ELI5 explanations** as requested.

SECTION A

Question 1

1(a) Microprocessor vs CPU & 8086 Block Diagram

What is a microprocessor?

A **microprocessor** is a single integrated circuit (IC) that contains the arithmetic, logic, and control units of a computer. It is the brain of a microcomputer system.

Difference between Microprocessor and CPU

Aspect	Microprocessor	CPU
Definition	A single-chip VLSI device	The processing unit (can be multiple chips in old computers)
Integration	Always a single IC	Could be multiple chips (e.g., 1970s CPUs used separate ALU, CU)
Examples	Intel 8085, 8086, ARM Cortex	CPU of a mainframe (multiple boards)
Modern usage	Terms are used interchangeably	Usually means the microprocessor itself

ELI5:
Microprocessor = one tiny chef inside a single kitchen.
CPU = the chef + all cooking tools (but now same thing).

Internal Block Diagram of Intel 8086 Microprocessor

text

```
┌──────────────────────────────────────────────────────────────────────────────
│                          INTEL 8086 CPU
│
│
┌──────────────────────────────────────────────────────────────────────────────┐
│
│  │                    BUS INTERFACE UNIT (BIU)
│    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│    │
│  │  │ Segment  │  │Instruction│ │ Address  │  │  Bus
│   │    │
│  │  │ Registers│  │  Queue   │ │ Generator │  │  Control
│   │    │
│  │  │ CS, DS,  │  │ (6 bytes) │ │          │  │
│   │    │
│  │  │ SS, ES   │  │          │ │          │  │
│   │    │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘
│    │
│  │                      │             │             │
│    │
│
└──────────────────────┼──────────────┼──────────────┼──────────┘
│
│                      │             │             │
│
│
┌──────────────────────┼──────────────┼──────────────┼──────────┐
│
│  │                    EXECUTION UNIT (EU)                 │
│    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐
┌──────▼──────┐ │    │
│  │  │ General  │  │   ALU    │  │   Flag   │  │  Control
│ │    │
│  │  │ Purpose  │  │ (Arith.  │  │ Register │  │  Circuitry
│ │    │
│  │  │ Registers│  │  & Logic)│  │          │  │
│ │    │
│  │  │ AX, BX,  │  │          │  │          │  │
│ │    │
│  │  │ CX, DX   │  └──────────┘  └──────────┘
└──────────────┘ │    │
│  │  └──────────┘
│    │
│
└──────────────────────────────────────────────────────────────────────────┘
│
│                      │             │
```

```
│
│                    Data Bus        Address/Control Bus
│
│                      (16-bit)
│
└──────────────────────────────────────────────────────────────────────────
```

Key Parts:

- **BIU** = Fetches instructions from memory (like a librarian getting books)
- **EU** = Executes instructions (like a student reading and solving)
- **Queue** = Pre-fetches next instructions (pipeline)

ELI5:
BIU = waiter bringing food before you finish current dish.
EU = your mouth chewing and tasting.

1(b) Bus, Bi-directional Data Bus, 64-bit Microprocessor

What is a bus?

A **bus** is a group of wires that transfers data, addresses, or control signals between components of a computer.

Three types:

1. **Data Bus** – carries data (bi-directional)
2. **Address Bus** – carries memory addresses (unidirectional from CPU)
3. **Control Bus** – carries control signals (RD, WR, IO/M, etc.)

Why is the data bus bi-directional?

Because data must flow **both ways**:

- **CPU** → **Memory/IO** (when writing data)
- **Memory/IO** → **CPU** (when reading data)

ELI5:
Data bus = two-way road. Cars (data) go from CPU to memory and back.
Address bus = one-way street. Only CPU sends addresses.

What is a 64-bit microprocessor?

A processor that can:

- Handle **64 bits of data** in one operation
- Has **64-bit data bus** (transfer 64 bits at a time)
- Has **64-bit general purpose registers**
- Can address **2^{64} bytes** of memory (theoretically)

Examples: Intel Core i7, AMD Ryzen, ARM Cortex-A (modern phones)

ELI5:
8-bit = small bucket (1 byte at a time)
16-bit = medium bucket (2 bytes)
32-bit = large bucket (4 bytes)
64-bit = huge bucket (8 bytes) – moves more water (data) per second.

Question 2

2(a) RISC vs CISC – Features, Differences with Block Diagram

RISC (Reduced Instruction Set Computer)

Features:

- Few instructions (≤ 100)
- Fixed instruction length (e.g., 32-bit)
- One instruction per clock cycle
- Large number of registers (32–64)
- Load/Store architecture (only LOAD/STORE access memory)

CISC (Complex Instruction Set Computer)

Features:

- Many instructions (200–500)
- Variable instruction length (1–15 bytes)
- Multiple clock cycles per instruction
- Fewer registers (8–16)
- Memory operands allowed in ALU instructions

Block Diagram Comparison

text

```
RISC Architecture                          CISC Architecture
┌──────────────────────┐
┌──────────────────────┐
│   Large Register     │                   │   Small Register
│
│   File (32-64)       │                   │   File (8-16)
│
└──────────────────────┘
└──────────────────────┘
           │                                          │
           ▼                                          ▼
┌──────────────────────┐
┌──────────────────────┐
│   Simple ALU         │                   │   Complex ALU
│
│   (only registers)   │                   │   (regs + memory)
│
└──────────────────────┘
└──────────────────────┘
           │                                          │
           ▼                                          ▼
┌──────────────────────┐
┌──────────────────────┐
│   Load/Store Unit    │                   │   Microcode ROM
│
│   (separate)         │                   │   (decodes complex
│
└──────────────────────┘                   │    instructions)
│

└──────────────────────┘
```

Differences Table

Feature	RISC	CISC
Instruction count	Few (~50-100)	Many (~200-500)
Instruction size	Fixed	Variable
Clock cycles per instruction	1	2-20

Feature	RISC	CISC
Registers	Many (32-64)	Few (8-16)
Memory access	Only LOAD/STORE	Many instructions
Hardware complexity	Simple	Complex
Power consumption	Low	High
Examples	ARM, MIPS, RISC-V	x86, 8086, Z80

ELI5:
RISC = sushi chef (does one thing perfectly, very fast, needs many plates)
CISC = buffet chef (one dish has many ingredients, but slower to prepare)

2(b) Compare 8085 and 8086 Processors

Feature	Intel 8085	Intel 8086
Year introduced	1976	1978
Data bus	8-bit	16-bit
Address bus	16-bit (64KB memory)	20-bit (1MB memory)
Registers	8-bit (can pair to 16-bit)	16-bit
Pipeline	No	Yes (BIU + EU)
Clock speed	3–6 MHz	5–10 MHz
Instruction set	CISC (simpler)	CISC (more complex)
Power supply	+5V single	+5V single
Number of pins	40	40
I/O ports	256 (8-bit address)	64KB (16-bit address)
Multiplication	No (software)	Yes (hardware, 8086 has MUL)

ELI5:
8085 = bicycle (8 bits at a time)
8086 = motorcycle (16 bits at a time, faster, can go further in memory)

Question 3

3(a) Absolute vs Partial Address Decoding

Absolute Address Decoding (Full Decoding)

Definition: All address lines are used to select a unique memory location. Each chip has **exactly one** address range.

Diagram:

text

```
A15 A14 A13 A12 ... A0
  |   |   |    |
  |   |   |    └──────── To chip address pins
  |   |   └─────────────── To decoder
  |   └──────────────────── To decoder
  └──────────────────────── To decoder
```

Example: 2KB RAM (needs 11 address lines A0–A10).
A11–A15 → decoder → unique chip select.
Result: One and only one address range.

Partial Address Decoding

Definition: Only some address lines are used. Unused lines are ignored → multiple addresses map to same chip.

Example: 2KB RAM, ignore A15.
Addresses 0x0000–0x07FF **and** 0x8000–0x87FF both access same chip.

Diagram:

text

```
Partial Decoding (A15 ignored)
┌──────────────────────────────────────┐
│ A15 (unused, tied to GND or VCC)     │
│   ↓                                  │
│ A14 A13 A12 ──→ Decoder              │
│   |    |    |                       │
│ A11–A0 ────→ Chip address pins       │
└──────────────────────────────────────┘
```

Comparison Table

Aspect	Absolute Decoding	Partial Decoding
Hardware	More (full decoder)	Less (simple gates)
Cost	Higher	Lower
Address space usage	Efficient	Wasted (aliasing)
Expansion	Easy (add more decoders)	Difficult
Debugging	Easy	Confusing (aliases)

ELI5:
Absolute = every house has a unique number.
Partial = whole street shares same number (postman confused).

3(b) Address Range for Memory Chips (from diagram)

From your diagram: A15, A14, A13 connected to 3-to-8 decoder.
Outputs Y0–Y7 go to MEM0–MEM7 chip select.

Assumption: Address lines A12–A0 go to each chip (8192 bytes = 8KB per chip).
A15–A13 = decoder select bits.

Decoder Truth Table

A15	A14	A13	Selected Chip	Start Address (A15..A0)	End Address
0	0	0	MEM0	0000H	1FFFH
0	0	1	MEM1	2000H	3FFFH
0	1	0	MEM2	4000H	5FFFH
0	1	1	MEM3	6000H	7FFFH
1	0	0	MEM4	8000H	9FFFH
1	0	1	MEM5	A000H	BFFFH
1	1	0	MEM6	C000H	DFFFH
1	1	1	MEM7	E000H	FFFFH

Formula: Each chip range = 8KB = 2^{13} bytes = 0x2000 bytes

Example for MEM0: Start = 0000H, End = 0000H + 2000H - 1 = 1FFFH

ELI5:
Decoder = reception desk. Each chip = a room. Address = room number.
A15–A13 = floor number. A12–A0 = room within floor.

Question 4

Given: 8-bit address bus (A0–A7), 8-bit data bus, IO/M, RD, WR signals.

4(a) I/O ports and Memory size directly addressable

I/O ports:

- With **isolated I/O** (IN/OUT instructions): 8-bit port address → **256 ports** (00H to FFH)

Memory size:

- 8-bit address bus → 2^8 = **256 bytes** maximum directly addressable

ELI5:
8 address lines = 8 switches. Each switch on/off = 256 combinations.
Like 8-bit binary lock has 256 possible codes.

4(b) Using 1KB memory with 8-bit address bus – Block Diagram

Problem: 8-bit address bus gives only 256 bytes, but we need 1KB (1024 bytes).

Solution: Use **bank switching** or **external latch** to extend addressing.

Block Diagram

text

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                   8-bit Microprocessor                                       │
│                                                                                              │
│   ┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────┐                                      │
│   │  A0-A7  │   │  D0-D7  │   │  IO/M   │   │  RD/WR  │                                      │
│   └────┬────┘   └────┬────┘   └────┬────┘   └────┬────┘                                      │
└────────┼─────────────┼─────────────┼─────────────┼──────────────────────────────────────────┘
         │             │             │             │
         ▼             ▼             ▼             ▼
┌──────────────┐  ┌──────────┐  ┌──────────────────────────┐
│ 74LS373      │  │ Data     │  │  Address Decoder         │
│ Latch        │  │ Bus      │  │                          │
│ (A8-A15)     │  │          │  │  IO/M ──┐                │
│              │  │          │  │  A7   ──┼──→ 3:8         │
│ Q0──→ A8     │  │          │  │  A6   ──┤    Decoder     │
│ Q1──→ A9     │  └──────────┘  │  A5   ──┤                │
│ ...          │       │        │         │                │
│ Q7──→ A15    │       │        │  Y0─────┼──→ Bank 0 CS   │
└──────────────┘       │        │  Y1─────┼──→ Bank 1 CS   │
       │               │        │  ...    │                │
       │               │        │  Y7─────┼──→ Bank 7 CS   │
       │               │        └──────────────────────────┘
       │               │                   │
       │               │                   ▼
       │               │          ┌──────────────────────┐
       │               │          │   1KB Memory Array   │
       │               │          │ (8 banks × 128 bytes)│
       │               └──────────┼                      │
       │                          │  Bank 0: 0x00-0x7F   │
       └──────────────────────────┼  Bank 1: 0x80-0xFF   │
                                  │  ...                 │
                                  └──────────────────────┘
```

How it works:

- Lower 7 bits (A0–A6) address 128 bytes within a bank
- Upper 1 bit (A7) + IO/M select bank via decoder
- External latch (74LS373) holds bank select

ELI5:
You have 8 bookshelves (banks), each with 128 pages.
You can only point to a page number (0-127), so you first say which shelf (bank), then which page.

4(c) Write program for tasks

Assumption: Memory is 1KB, organized as 8 banks of 128 bytes each.
Bank select via port 00H (output bank number), then use MOV instruction.

(i) Move 03 to memory address 102 (decimal)

102 decimal = 0x66 (hex)

assembly

```
        MVI A, 00H      ; Select bank 0 (addresses 0-127)
        OUT 00H         ; Output bank select
        MVI A, 03H      ; Value to store
        MOV 66H, A      ; Store 03 at memory address 66H (102
decimal)
        HLT
```

(ii) Move 04 to memory address F0 (hex)

F0 hex = 240 decimal → this is in bank 1 (since bank 0: 0-127, bank 1: 128-255)
Offset within bank = F0H - 80H = 70H

assembly

```
        MVI A, 01H      ; Select bank 1 (addresses 128-255)
        OUT 00H         ; Output bank select
        MVI A, 04H      ; Value to store
        MOV 70H, A      ; Store 04 at offset 70H within bank 1
        HLT
```

Alternative simpler method (if memory is 1KB continuous using external hardware):

If hardware maps 1KB as continuous (requires 10 address lines A0–A9), you need two MOV instructions because 8-bit CPU can't directly address 10 bits.

assembly

```
; For address 102 (0x66) - still within low 256 bytes
MVI A, 03H
MOV 66H, A

; For address 0xF0 (240) - still within low 256 bytes
MVI A, 04H
MOV F0H, A
HLT
```

ELI5:
MOV = "put this value into this mailbox number".
OUT = "tell the post office which shelf to use".

Quick Revision Table for Section A

Term	ELI5
Microprocessor	One tiny chef on a chip
Bus	Highway for data/addresses
64-bit	8-lane highway (vs 1-lane for 8-bit)
RISC	Fast, simple, many registers
CISC	Complex, slower, fewer registers
Absolute decoding	Each house has unique number
Partial decoding	Whole street has same number
8085	8-bit bicycle
8086	16-bit motorcycle

Good luck tomorrow! Focus on diagrams (8086 block diagram, decoder connections), tables (comparisons), and show your calculations for address ranges. You will do great! 💯

MPC22_2.jpg

JPG 2.95MB

Here is the complete, exam-oriented solution for **Section B (Questions 5–8)** from your MPC22_2.jpg paper.
I have included **diagrams, tables, real-world examples, and ELI5 explanations** as requested.

SECTION B

Question 5

5(a) Advantages of ARM processors over x86 processors

Aspect	ARM	x86 (Intel/AMD)
Architecture	RISC (Reduced Instruction Set)	CISC (Complex Instruction Set)
Power consumption	Very low (mW to few W)	High (15W to 150W)
Instruction length	Fixed (32-bit or 16-bit Thumb)	Variable (1–15 bytes)
Pipeline stages	3–7 stages (simple)	14–19 stages (complex)
License model	IP licensing (anyone can make)	Proprietary (only Intel/AMD)
Cost	Lower (royalty-based)	Higher
Performance per watt	Excellent	Poor
Heat generation	Low	High (needs fan)
Target devices	Mobile, embedded, IoT, tablets	Desktops, servers, gaming
Examples	Snapdragon, Apple M series, Cortex-M	Core i7, Ryzen, Xeon

Key Advantages (Exam focus):

1. **Energy Efficiency** – ARM can run for days on battery; x86 laptops last hours

2. **Lower cost** – ARM chips cheaper to manufacture
3. **Smaller die size** – Fits in tiny devices (smartwatches, sensors)
4. **No fan required** – Passive cooling possible
5. **Better for embedded** – Real-time capabilities, deterministic interrupts
6. **Scalability** – From Cortex-M0 (simple) to Cortex-A78 (powerful)

Real-world example:
iPhone (ARM) lasts 20 hours video playback vs Intel laptop ~6 hours.

ELI5:
ARM = bicycle (uses little energy, perfect for short trips)
x86 = monster truck (powerful but drinks gas, needs big garage)

5(b) What is a Microcontroller? Categories of MCU

Microcontroller Definition

A **microcontroller (MCU)** is a single-chip computer containing:

- CPU core
- RAM (data memory)
- ROM/Flash (program memory)
- I/O ports
- Timers, ADC, communication peripherals (UART, SPI, I²C)

ELI5:
Microprocessor = engine only (needs external fuel tank, wheels, chassis)
Microcontroller = complete car (engine + fuel + wheels + dashboard in one chip)

Categories of MCU (by bit size)

Category	Bit size	Examples	Applications
4-bit	4	TMS1000, Sharp SM series	Calculators, toys, remote controls
8-bit	8	8051, PIC, AVR (Arduino Uno)	Keyboards, fans, washing machines
16-bit	16	MSP430, PIC24	Medical devices, automotive
32-bit	32	STM32 (ARM Cortex-M), ESP32	IoT, drones, robotics, phones
64-bit	64	Rare in MCU (some automotive)	High-end automotive, AI edge

Categories by Architecture

Type	Description	Example
Harvard	Separate code/data buses	AVR, 8051, PIC
Von Neumann	Shared code/data bus	ARM Cortex-M (some)
CISC	Complex instructions	8051
RISC	Reduced instructions	ARM, PIC, AVR

ELI5:
4-bit = pocket calculator (adds 0-15)
8-bit = simple robot (0-255)
32-bit = smartphone brain (0 to 4 billion)

5(c) Role and Responsibility of an STM32 Developer

Role

An STM32 developer writes firmware (software) for STM32 microcontrollers to control hardware peripherals.

Key Responsibilities

Responsibility	Description
1. Peripheral Configuration	Set up GPIO, ADC, timers, UART, SPI, I²C using HAL or LL libraries

Responsibility	Description
2. Interrupt Handling	Write ISRs (Interrupt Service Routines) for real-time response
3. Power Management	Use sleep modes (Stop, Standby) for battery devices
4. Communication Protocols	Implement I²C, SPI, USART, CAN, USB
5. Debugging	Use ST-Link, J-Link, printf via SWO, breakpoints
6. Bootloader Development	Write code for OTA (Over-The-Air) updates
7. RTOS Integration	Use FreeRTOS, Zephyr, or bare-metal scheduling
8. Documentation	Write comments, hardware abstraction, user manuals

Required Tools

text

```
┌─────────────────────────────────────────────────────────┐
│                  STM32 Development Stack                 │
├─────────────────────────────────────────────────────────┤
│  IDE: STM32CubeIDE, Keil, IAR, VS Code                   │
│  HAL: Hardware Abstraction Layer (ST provided)           │
│  Config: STM32CubeMX (pin/clock setup)                   │
│  Debugger: ST-Link, J-Link, Black Magic Probe            │
│  Programmer: STM32CubeProgrammer                         │
└─────────────────────────────────────────────────────────┘
```

Real-world example:
An STM32 developer at a drone company writes code to:

- Read gyroscope (I²C)
- Control motor speed (PWM)
- Send telemetry (UART)
- Enter low-power mode when idle

ELI5:
STM32 developer = chef who cooks meals (firmware) using a special kitchen (STM32) with many appliances (peripherals).

Question 6

6(a) Shift/Rotate Instructions with Diagram (ASR, LSR, LSL, ROR, RRX)

Assume initial **R0 = 0b 1100 1011** (8-bit for clarity, ARM uses 32-bit)

(i) ASR #3 (Arithmetic Shift Right by 3)

Preserves sign bit (MSB)

text

```
Before: 1 1 0 0 1 0 1 1  (MSB=1 → negative)
After:  1 1 1 1 1 0 0 1  (shift right 3, fill with 1s)

Diagram:
MSB→|1|1|0|0|1|0|1|1| ←LSB
     | ↓  ↓  ↓
     | 1|1|1|1|1|0|0|1|
        ↑ ↑ ↑ sign bit (1) fills
```

(ii) LSR #3 (Logical Shift Right by 3)

Shift right, fill with zeros

text

```
Before: 1 1 0 0 1 0 1 1
After:  0 0 0 1 1 0 0 1

Diagram:
┌──────────────────────────┐
│1|1|0|0|1|0|1|1|          │
│ ↓  ↓  ↓                  │
│0|0|0|1|1|0|0|1| (zeros in)│
└──────────────────────────┘
```

(iii) LSL #3 (Logical Shift Left by 3)

Shift left, fill LSB with zeros

text

```
Before: 1 1 0 0 1 0 1 1
After:  0 1 0 1 1 0 0 0

Diagram:
┌─────────────────────────┐
│1│1│0│0│1│0│1│1│         │
│     ↓  ↓  ↓             │
│0│1│0│1│1│0│0│0│ (zeros in)│
└─────────────────────────┘
```

(iv) ROR #3 (Rotate Right by 3)

Bits shifted out from LSB come back into MSB

text

```
Before: 1 1 0 0 1 0 1 1
After:  0 1 1 1 1 0 0 1
(The 3 LSB bits 011 move to front)

Diagram:
        ┌─────────────────┐
        │                 │
        ▼                 │
┌───────────────────────┬─┐
│1│1│0│0│1│0│1│1│ ──────┘ │
│        / / /            │
│      ┌──────────────────┘
│      ▼
│0│1│1│1│1│0│0│1│
└─────────────────┘
```

(v) RRX #3 (Rotate Right through Carry by 3)

Rotates **through the Carry flag (C)** – treats C as an extra bit

text

```
Initial: C=0, R0=1 1 0 0 1 0 1 1

After RRX #3 (rotate 3 times through carry):
Step 1: C=1 (LSB), R0 = 0 1 1 0 0 1 0 1
Step 2: C=1, R0 = 1 0 1 1 0 0 1 0
Step 3: C=0, R0 = 0 1 0 1 1 0 0 1

Final: C=0, R0 = 0 1 0 1 1 0 0 1
```

RRX Diagram:

text

```
Before:  [C] [R7 R6 R5 R4 R3 R2 R1 R0]
          0    1  1  0  0  1  0  1  1

After 1 rotation:
          1    0  1  1  0  0  1  0  1
          ↑         ↑
    old LSB→C   C→MSB
```

ELI5:

- **ASR** = Slide right, keep the sign (like pushing a negative number)
- **LSR** = Slide right, push zeros (like emptying a conveyor belt)
- **LSL** = Slide left, push zeros (faster, multiplies by 2)
- **ROR** = Circular conveyor belt (bits don't fall off)
- **RRX** = Conveyor belt with an extra parking spot (Carry flag)

6(b) LDR and STR Instructions with Example

LDR (Load Register)

Loads data from memory into a register.

Syntax: `LDR Rd, [Rn, #offset]`

STR (Store Register)

Stores data from a register into memory.

Syntax: `STR Rd, [Rn, #offset]`

Addressing Modes Table

Mode	Example	Meaning
Immediate offset	`LDR R0, [R1, #4]`	R0 = mem[R1 + 4]
Pre-indexed	`LDR R0, [R1, #4]!`	R1 = R1+4, then R0=mem[R1]
Post-indexed	`LDR R0, [R1], #4`	R0=mem[R1], then R1=R1+4
Register offset	`LDR R0, [R1, R2]`	R0 = mem[R1 + R2]

Complete Example

assembly

```
; Assume R1 = 0x20000000 (memory address)

LDR R0, [R1]        ; R0 = value at 0x20000000
ADD R0, R0, #1      ; R0 = R0 + 1
STR R0, [R1]        ; store updated value back

; Another example with offset
LDR R2, [R1, #8]    ; R2 = value at 0x20000008
STR R3, [R1, #12]   ; store R3 at 0x2000000C
```

Real-world example (LED toggle)

assembly

```
; GPIO output data register at 0x40010C0C
LDR R0, =0x40010C0C  ; load address
LDR R1, [R0]         ; read current value
EOR R1, R1, #1       ; toggle bit 0
STR R1, [R0]         ; write back → LED toggles
```

ELI5:
LDR = "Look in this mailbox, tell me what's inside"
STR = "Put this letter into that mailbox"

Question 7

7(a) Load R0 with contents from memory address in R1

Given: Memory location 0x3F02F1D4 stored in R1.
Load contents into R0.

assembly

```
LDR R0, [R1]        ; R0 = *(uint32_t*)R1
```

Explanation:

- `[R1]` means "the memory address pointed to by R1"
- LDR loads 32 bits (word) from that address into R0

ELI5:
R1 has a street address written on a sticky note. LDR goes to that house, opens the door, and brings back what's inside.

7(b) Count 1's in R0 (Bit Counting Problems)

(i) Count all 1's in R0 → store in R2

Method 1: Loop and shift (simple)

assembly

```
        MOV R2, #0          ; R2 = counter
        MOV R3, #32         ; 32 bits to check
LOOP1:  ANDS R1, R0, #1     ; check LSB (sets Z flag)
        ADDNE R2, R2, #1    ; if LSB=1, increment
        LSRS R0, R0, #1     ; shift right, LSB lost
        SUBS R3, R3, #1     ; decrement bit counter
        BNE LOOP1           ; loop until 32 bits done
```

Method 2: Using `RBIT` + `POPCNT` (Cortex-M4+ has `__builtin_popcount`)

assembly

```
        ; Using ARMv7-M instruction (if available)
        RBIT R0, R0         ; reverse bits
        ; Count ones using popcount (not native, so loop
needed)
```

(ii) Count 1's in lower 16 bits (bit 0–15) → store in R3

assembly

```
        MOV R3, #0          ; counter
        MOV R1, #16         ; 16 bits
        MOV R2, R0          ; copy R0
LOOP2:  ANDS R4, R2, #1     ; check LSB
        ADDNE R3, R3, #1    ; increment if 1
        LSRS R2, R2, #1     ; shift right
        SUBS R1, R1, #1
        BNE LOOP2
```

Alternative (masking method):

assembly

```
        AND R0, R0, #0xFFFF ; clear upper 16 bits
        ; then use same loop as (i)
```

(iii) Count 1's at even bit positions (0,2,4...30) → store in R4

assembly

```
        MOV R4, #0          ; counter
        MOV R1, #0          ; bit index
        MOV R2, R0          ; copy R0
LOOP3:  ANDS R3, R2, #1     ; check bit
        ADDNE R4, R4, #1    ; count if 1
        LSRS R2, R2, #2     ; shift right by 2 (skip odd bits)
        ADD R1, R1, #2
        CMP R1, #32
        BLT LOOP3
```

(iv) Count 1's at specific bit positions (list given) → store in R5

Bit positions: 0,1,3,6,11,14,17,21,22,23,28,31

assembly

```
        MOV R5, #0          ; counter
        LDR R1, =bit_mask   ; load mask from memory

        ; Method: AND with mask, then count bits in result
        LDR R2, =0x0001     ; bit 0 mask
        LDR R3, =0x0002     ; bit 1 mask
        LDR R4, =0x0008     ; bit 3 mask
        ; ... load all masks

        ; Check each bit position
        TST R0, #0x01       ; bit 0
        ADDEQ R5, R5, #0    ; if zero, no add
        ADDNE R5, R5, #1    ; if one, add 1

        TST R0, #0x02       ; bit 1
        ADDNE R5, R5, #1

        TST R0, #0x08       ; bit 3
        ADDNE R5, R5, #1

        ; ... continue for all 12 positions
```

Optimized method (single mask):

assembly

```
        ; Precompute mask = 0x80204C46 (check bits)
        ; bit31=1, bit28=1, bit23=1, bit22=1, bit21=1,
        ; bit17=1, bit14=1, bit11=1, bit6=1, bit3=1, bit1=1,
bit0=1
        LDR R1, =0x80204C46  ; mask value
        AND R0, R0, R1       ; keep only target bits
        ; Now count 1's in R0 (same as part i)
        MOV R5, #0
        MOV R2, #32
LOOP4:  ANDS R3, R0, #1
        ADDNE R5, R5, #1
        LSRS R0, R0, #1
        SUBS R2, R2, #1
        BNE LOOP4
```

ELI5:
Counting 1's = counting how many light switches are ON.
Even positions = every second switch (0,2,4...).
Specific positions = checking only certain switches (like only the important ones).

7(c) Key Features of Advanced Peripheral Bus (APB) of STM32F103x8

The **APB** (Advanced Peripheral Bus) is part of ARM's AMBA (Advanced Microcontroller Bus Architecture).

Features Table

Feature	Description
1. Lower speed	Runs at lower frequency than AHB (typically HCLK/2 or HCLK/4)
2. Low power	Designed for low-power peripherals (timers, USART, I²C, SPI)
3. Two versions	APB1 (36 MHz max), APB2 (72 MHz max for STM32F103)
4. Simple interface	No pipeline, single-cycle access
5. Address decoding	Simple 1-to-1 mapping
6. Peripheral access	All non-critical peripherals connected to APB

APB Peripherals in STM32F103

APB1 (max 36 MHz)	APB2 (max 72 MHz)
USART2, USART3	USART1
I2C1, I2C2	SPI1
SPI2, SPI3	GPIOA, GPIOB, GPIOC
TIM2, TIM3, TIM4	TIM1 (advanced)
WWDG, IWDG	ADC1, ADC2
PWR, BKP	EXTI, AFIO

Diagram:

text

```
┌──────────────────────────────────────────────────────────┐
│                        STM32F103x8                       │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────┐          AHB (72 MHz)          ┌──────────┐│
│  │ Cortex   │◄──────────────────────────────►│  Flash   │
│  │   M3     │                                │  SRAM    │
│  └──────────┘                                └──────────┘
│       │                                                  │
│       │ AHB Bus                                          │
│       ▼                                                  │
│  ┌──────────┐          APB1 Bridge           ┌──────────┐
│  │   DMA    │                                │ APB1     │
│  └──────────┘                                │ (36 MHz) │
│                                              │ TIM2..4  │
│  ┌──────────┐          APB2 Bridge           │ USART2/3 │
│  │   GPIO   │◄──────────────────────────────►│ I2C      │
│  └──────────┘                                └──────────┘
│                                              ┌──────────┐
│                                              │ APB2     │
│                                              │ (72 MHz) │
│                                              │ GPIO     │
│                                              │ USART1   │
│                                              │ ADC      │
│                                              └──────────┘
└──────────────────────────────────────────────────────────┘
```

ELI5:
AHB = highway (fast, for CPU and memory)
APB = local street (slower, for sensors and buttons)
APB1 = residential street (slower, 36 km/h)
APB2 = main road (faster, 72 km/h)

Question 8

8(a) Key Functions of the RCC Module of STM32F103x8

RCC = Reset and Clock Control

Key Functions

Function	Description
1. Clock source selection	Choose HSI, HSE, PLL, LSI, LSE
2. System clock configuration	Set SYSCLK up to 72 MHz

Function	Description
3. Peripheral clock enable/disable	Turn on/off clocks to save power
4. Clock prescalers	AHB, APB1, APB2 dividers
5. Reset generation	System reset, peripheral reset
6. Clock output (MCO)	Output internal clock to pin
7. Ready flags	Check if clock is stable (HSIRDY, HSERDY, PLLRDY)

RCC Register Map

Register	Purpose
`RCC_CR`	Clock control (HSI/HSE/PLL on/off, ready flags)
`RCC_CFGR`	Clock configuration (prescalers, MCO)
`RCC_CIR`	Clock interrupt register
`RCC_APB2ENR`	APB2 peripheral clock enable
`RCC_APB1ENR`	APB1 peripheral clock enable
`RCC_AHBENR`	AHB peripheral clock enable
`RCC_BDCR`	Backup domain control (LSE, RTC)
`RCC_CSR`	Control/status register (reset flags)

Typical initialization code:

c

```c
RCC->CR |= RCC_CR_HSEON;         // Enable HSE
while(!(RCC->CR & RCC_CR_HSERDY)); // Wait for HSE ready
RCC->CFGR |= RCC_CFGR_PLLSRC_HSE; // PLL source = HSE
RCC->CFGR |= RCC_CFGR_PLLMULL9;   // PLL x9 = 8MHz*9=72MHz
RCC->CR |= RCC_CR_PLLON;          // Enable PLL
while(!(RCC->CR & RCC_CR_PLLRDY)); // Wait for PLL
RCC->CFGR |= RCC_CFGR_SW_PLL;     // Select PLL as system
clock
```

ELI5:
RCC = building's electrical panel. It decides:

- Which power plant to use (HSI, HSE, PLL)
- Which rooms get electricity (peripherals)
- How much voltage (prescalers)
- Emergency shutoff (reset)

8(b) Bit-band alias address calculation

Given: Peripheral region. Bit 0 of RCC_CR at address 0x40021000

Formula for alias address:

text

```text
Alias = 0x42000000 + (byte_offset × 32) + (bit_number × 4)
```

Where: `byte_offset = target_address - 0x40000000`

Step 1: byte_offset = 0x40021000 − 0x40000000 = 0x21000

Step 2: Alias = 0x42000000 + (0x21000 × 32) + (0 × 4)

Step 3: 0x21000 × 32 = 0x21000 × 0x20 = 0x420000 (calculate: 0x21000 << 5)

Step 4: 0x42000000 + 0x420000 = 0x42420000

Add bit offset: 0 × 4 = 0

Answer: 0x42420000

Verification: This matches your 8(c) R4 value! (They gave R4=0x42420000)

ELI5:
Bit 0 of RCC_CR now has its own private mailbox at 0x42420000.
Writing to that address only changes bit 0, leaving other bits untouched.

8(c) STRD instruction execution

`STRD R3, [R4]` stores **two 32-bit values**: R3 and R4 into memory starting at address in R4.

Given R4 = 0x42420000

Case	R3 value	Memory at 0x42420000	Memory at 0x42420004
(i)	0xFF	0x000000FF	0x42420000
(ii)	0x21	0x00000021	0x42420000
(iii)	0x00	0x00000000	0x42420000
(iv)	0x10	0x00000010	0x42420000

Important notes:

- STRD stores **R3** to `[R4]` (first word)
- STRD stores **R4** (the original address) to `[R4+4]` (second word)
- Values are zero-extended to 32 bits

ELI5:
STRD = "Store Double" – packs two suitcases (R3 and R4) into two consecutive lockers starting at locker number R4.

8(d) Explain STM32 code

c

```c
RCC->CR |= RCC_CR_HSEON;                    // Line 1
while (!(RCC->CR & RCC_CR_HSERDY));         // Line 2
```

Line-by-line explanation

Line 1: `RCC->CR |= RCC_CR_HSEON;`

- `RCC_CR` = Clock Control Register (address 0x40021000)
- `RCC_CR_HSEON` = bit 16 (HSE Oscillator Enable bit)
- `|=` sets bit 16 to 1 without changing other bits
- **Effect:** Turns on the external crystal oscillator (HSE)

Line 2: `while (!(RCC->CR & RCC_CR_HSERDY));`

- `RCC_CR_HSERDY` = bit 17 (HSE Ready flag)
- `RCC->CR & RCC_CR_HSERDY` checks if bit 17 = 1
- `!` means "wait while NOT ready"
- **Effect:** Waits (busy loop) until HSE oscillator stabilizes

Timing diagram

text

```text
HSEON bit set ──┐
               │
               ▼
        ┌──────────────┐
        │ HSE starting │  (few ms)
        │ oscillating  │
        └──────────────┘
               │
               ▼
HSERDY bit = 1 ──┼──► Exit while loop
               │
               ▼
          Continue execution
```

Complete context (typical HSE initialization)

c

```c
// Enable HSE
RCC->CR |= RCC_CR_HSEON;
while (!(RCC->CR & RCC_CR_HSERDY));

// Configure PLL to use HSE
RCC->CFGR |= RCC_CFGR_PLLSRC_HSE;
RCC->CFGR |= RCC_CFGR_PLLMULL9;  // 8MHz * 9 = 72MHz

// Enable PLL
RCC->CR |= RCC_CR_PLLON;
while (!(RCC->CR & RCC_CR_PLLRDY));

// Select PLL as system clock
RCC->CFGR |= RCC_CFGR_SW_PLL;
```

ELI5:
Line 1 = Press the "start" button on the backup generator (HSE)
Line 2 = Wait and watch the "ready" light. Don't proceed until it's fully on.

Quick Revision Table for Section B

Term	ELI5
ARM vs x86	Bicycle vs Monster truck
MCU	Complete computer on one chip
STM32 Developer	Chef for STM32 kitchen
LDR	"Look inside this mailbox"
STR	"Put this letter in mailbox"
ASR	Slide right, keep negative sign
RRX	Conveyor belt with extra parking spot
APB	Local street for slow devices
RCC	Building's electrical panel
Bit-banding	Give each bit its own mailbox
STRD	Pack two suitcases in two lockers
HSEON	"Start backup generator"

Good luck tomorrow! Key points to remember:

- Draw diagrams for shift/rotate instructions
- Show bit counting loops clearly
- Use tables for comparisons (ARM vs x86, APB features)
- Show step-by-step address calculation for bit-band

You've got this! 🚀

MPC21.jpg

JPG 3.24MB

Here is the complete, exam-oriented solution for **Section A & B (Questions 1–8)** from your MPC21.jpg paper.
I have included **diagrams, tables, step-by-step calculations, real-world examples, and ELI5 explanations** as requested.

SECTION A

Question 1

1(a) 8086 Architectural Diagram – BIU and EU Functions

Block Diagram of Intel 8086

text

```
┌──────────────
```

- **EU** = Your mouth (chews and swallows the food)
- **Queue** = Extra plates on table so waiter can bring more before you finish

1(b) Address Capability & I/O Lines of 8086

(i) Address Capability

- **20 address lines** (A0–A19)
- Can address **2^{20} = 1,048,576 bytes = 1 MB** of memory

(ii) I/O Lines Accessible

- **16 address lines** for I/O (A0–A15)
- Can access **2^{16} = 65,536** I/O ports (64 KB I/O space)

ELI5:
Memory = 1 million mailboxes
I/O ports = 65 thousand special mailboxes connected to devices

1(c) Steps in Physical Address Generation in 8086

Formula:

`Physical Address = (Segment Register × 10H) + Offset`

Step-by-step process

text

```
Step 1: Segment register (16-bit) → shift left by 4 bits
(multiply by 16)
Example: CS = 0A32H
        0A32H × 10H = 0A320H

Step 2: Add offset (16-bit)
Example: IP = 0028H
        0A320H + 0028H = 0A348H

Step 3: Result is 20-bit physical address
        0A348H = 0000 1010 0011 0100 1000 binary
```

Diagram

text

```
┌──────────────┐      ┌──────────────┐
│ Segment Reg  │      │   Offset     │
│ (16-bit)     │      │   (16-bit)   │
│   CS = 0A32  │      │   IP = 0028  │
└──────────────┘      └──────────────┘
       │                     │
       ▼                     │
┌──────────────┐             │
│ Shift Left 4 │             │
│ 0A32 → 0A320 │             │
└──────────────┘             │
       │                     │
       └─────────────────────┘
                ▼
          ┌──────────────┐
          │    ADD       │
          │ 0A320 + 0028 │
          │      =       │
          │   0A348H     │
          └──────────────┘
                │
                ▼
          ┌──────────────┐
          │ 20-bit Physical│
          │   Address    │
          └──────────────┘
```

ELI5:
Segment × 16 = "street number × 16"
Add offset = "add house number"
Result = full address where the data lives

Question 2

2(a) Multiplexing of Address and Data Buses in 8086

Why Multiplex?

- 8086 has only 40 pins
- Needs 20 address + 16 data = 36 pins just for buses
- **Solution:** Share pins (multiplex)

How it works

text

```
Address/Data multiplexing on pins AD0-AD15

┌──────────────────────────────────────────────────────┐
│                    T1 (First clock)                  │
│  ┌──────────────────────────────────────────────┐    │
│  │ A0-A15 (Address) appear on AD0-AD15 pins     │    │
│  │ A16-A19 appear on A16/S3-A19/S6 pins         │    │
│  └──────────────────────────────────────────────┘    │
│                         │                            │
│                         ▼                            │
│                    T2, T3, T4                        │
│  ┌──────────────────────────────────────────────┐    │
│  │ D0-D15 (Data) appear on same AD0-AD15 pins   │    │
│  └──────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────┘
```

Demultiplexing Circuit

text

```
8086 pins                    External Circuit

┌────────┐                   ┌──────────────────────┐
│  AD0   ├───────────────────┼──┬───────────────────┤
│  AD1   ├───────────────────┼──┼┬──────────────────┤
│  ...   │                   │  │└                  │
│  AD15  ├───────────────────┼──┼┼──────→ Data Bus (16) │
└────────┘                   │  │└                  │
                             │  │  ┌──────────────┐  │
┌────────┐                   │  └──┤ 74LS373      │  │
│  ALE   ├───────────────────┼─────┤ Latch        │  │
└────────┘                   │     │              │  │
                             │     │ Q0──────→ A0 │  │
                             │     │ Q1──────→ A1 │  │
                             │     │ ...          │  │
                             │     │ Q15─────→ A15│  │
                             │     └──────────────┘  │
                             └──────────────────────┘
```

Working:

1. **T1 cycle:** ALE = 1 → latch stores address from AD0–AD15
2. **T2–T4 cycles:** ALE = 0 → latch holds address, AD pins now carry data

ELI5:
Multiplexing = a single road used by cars (address) and trucks (data) at different times.

ALE = traffic light that tells you when cars are coming vs trucks.

2(b) Function of 8086 Pins

Pin	Function	ELI5
MN/MX	**Minimum/Maximum mode select** MN=1 → single processor mode MX=0 → multiprocessor mode	Single player vs Multiplayer mode in a game
READY	**Wait state control** READY=0 → CPU inserts wait states READY=1 → CPU continues	"Slow down" signal from slow memory
ALE	**Address Latch Enable** ALE=1 → address on AD bus ALE=0 → data on AD bus	Flashing light: "Now address, now data"
DT/R	**Data Transmit/Receive** DT/R=1 → CPU sends data (write) DT/R=0 → CPU receives data (read)	Traffic sign: "One way this direction"

Real-world example:

- **READY** = Like telling a fast talker "Hold on, I'm still writing"
- **ALE** = A teacher pointing: "Now read the question, now write the answer"

2(c) Memory Segmentation of 8086 Processor

What is Segmentation?

Dividing the 1 MB memory into **logical blocks** called **segments** (64 KB each).

Segment Registers

Register	Purpose	Default Offset
CS	Code Segment (instructions)	IP (Instruction Pointer)

Register	Purpose	Default Offset
DS	Data Segment (variables)	SI, DI, BX
SS	Stack Segment (stack data)	SP (Stack Pointer)
ES	Extra Segment (string destination)	DI

Advantages of Segmentation

Advantage	Explanation
1. Relocation	Programs can be loaded anywhere in memory
2. Protection	Segments can be protected (not in 8086, but later CPUs)
3. Multiple programs	Each program can have its own segments
4. Code/Data separation	Code, data, stack in different segments
5. 1MB address with 16-bit registers	20-bit address from 16-bit registers

Diagram: Segmented Memory View

text

```
Memory Address (hex)    Logical View
00000 ┌─────────────────┐
      │                 │
      │   Code Segment  │ ← CS:IP
      │   (64KB max)    │
      │                 │
10000 ├─────────────────┼┐
      │                 │ │
      │   Data Segment  │ │ DS:SI
      │   (64KB max)    │ │
      │                 │ │
20000 ├─────────────────┼┘
      │                 │
      │   Stack Segment │ ← SS:SP
      │   (64KB max)    │
      │                 │
30000 ├─────────────────┼┐
      │                 │ │
      │   Extra Segment │ │ ES:DI
      │   (64KB max)    │ │
      └─────────────────┘ ┘
```

ELI5:
Segmentation = Library with different rooms:

- CS = Reading room (code)
- DS = Book stacks (data)
- SS = Break room (stack)
- ES = Copy room (extra)

Question 3

3(a) Byte and Word Data Read from Odd/Even Address

8086 Memory Organization

text

```
Memory Bank Structure
┌──────────────────────────────────────┐
│         Even Bank (D0-D7)            │
│  Address: 1000H (low byte)           │
│           1002H (low byte)           │
│           ...                        │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│         Odd Bank (D8-D15)            │
│  Address: 1001H (high byte)          │
│           1003H (high byte)          │
│           ...                        │
└──────────────────────────────────────┘
```

Case 1: Word at Even Address (e.g., 1000H)

text

```
One bus cycle:
┌──────────┐      ┌──────────────────┐
│ 8086     │      │ Even: 1000H (low)│
│          │──────→│ Odd:  1001H (high)│
└──────────┘      └──────────────────┘
```

Case 2: Word at Odd Address (e.g., 1001H)

text

```
Two bus cycles:
Cycle 1: ┌──────────┐      ┌──────────────────┐
         │ 8086     │──────→│ Odd: 1001H (low) │
         └──────────┘      └──────────────────┘

Cycle 2: ┌──────────┐      ┌──────────────────┐
         │ 8086     │──────→│ Even: 1002H (high)│
         └──────────┘      └──────────────────┘

Then CPU reassembles the word in internal register
```

ELI5:
Even address = house numbers on same side of street (one trip)
Odd address = numbers on opposite sides (two trips, then glue together)

3(b) Memory Models of 8086 Processor

Model	Code Segments	Data Segments	Use Case
TINY	1	1 (CS=DS=SS=ES)	.COM files (<64KB)
SMALL	1	1	Most programs
MEDIUM	Many	1	Large code, small data
COMPACT	1	Many	Small code, large data
LARGE	Many	Many	Both large
HUGE	Many	Many	Arrays >64KB allowed

ELI5:

- TINY = Studio apartment
- SMALL = 1-bedroom apartment
- MEDIUM = Many rooms for books (code), 1 room for clothes (data)
- LARGE = Mansion with many rooms for everything

3(c) Determine Status Flags After ADD AX, BX

Given:
AX = FFFFH
BX = FFFFH
Instruction: `ADD AX, BX`

Calculation:

text

```
   FFFF
+  FFFF
= 1 FFFE  (1 is carry out)

In binary: 1111 1111 1111 1111
         + 1111 1111 1111 1111
         = 1 1111 1111 1111 1110
           ↑
        Carry (1)
```

Result in AX = FFFEH (because 16-bit result)

Flag Status:

Flag	Value	Explanation
CF (Carry)	1	There is a carry out of MSB
PF (Parity)	0	FFFE has 15 ones (odd → parity=0)
AF (Aux Carry)	1	Carry from bit 3 to bit 4 (FFFF+FFFF)
ZF (Zero)	0	Result ≠ 0
SF (Sign)	1	MSB of result is 1 (negative)
OF (Overflow)	0	Both positive → sum positive (no overflow)

ELI5:
Two large numbers added = result too big for 16-bit box.
The extra 1 falls out (Carry flag = 1). The result FFFE is negative (Sign flag = 1).

Question 4

4(a) Role of Segment and Offset Registers

Segment Registers

Register	Role	Used With
CS	Points to current code segment	IP (Instruction Pointer)
DS	Points to current data segment	BX, SI, DI, immediate
SS	Points to current stack segment	SP, BP
ES	Points to extra data segment	DI (string destination)

Offset Registers

Register	Role	Default Segment
IP	Instruction Pointer (next instruction)	CS
SP	Stack Pointer (top of stack)	SS
BP	Base Pointer (stack frame)	SS
SI	Source Index (string source)	DS
DI	Destination Index (string dest)	ES
BX	Base register (data pointer)	DS

Diagram:

text

```
┌──────────────────────────────────────────────────────────┐
│                    Memory (1MB)                          │
│                                                          │
│  CS:IP ──────────────────→ ┌──────────────────┐          │
│                            │  Code (instructions)│        │
│                            └──────────────────┘          │
│                                                          │
│  DS:BX ──────────────────→ ┌──────────────────┐          │
│  DS:SI                     │  Data (variables) │         │
│                            └──────────────────┘          │
│                                                          │
│  SS:SP ──────────────────→ ┌──────────────────┐          │
│                            │  Stack            │         │
│                            └──────────────────┘          │
└──────────────────────────────────────────────────────────┘
```

ELI5:
Segment = Library section (fiction, non-fiction, reference)
Offset = Shelf number within that section

4(b) Find Segment for Given Physical Address

Given:
Physical address = 80FB0DH
Offset = BF0D2H

Formula:
`Physical = (Segment × 10H) + Offset`

Step 1: Rearrange: `Segment × 10H = Physical - Offset`

Step 2: Calculate:
80FB0DH - BF0D2H

text

```
  80FB0D
-  BF0D2
=  ?????
```

Convert to decimal or do hex subtraction:

80FB0D = 0x80FB0D
BF0D2 = 0x0BF0D2

text

```
  0x80FB0D
- 0x0BF0D2
= 0x750A3B
```

Step 3: Divide by 10H (shift right 4 bits):
0x750A3B ÷ 0x10 = **0x750A3** (remainder B)

Answer: Segment = **750A3H**

Check:
750A3H × 10H = 750A30H
750A30H + BF0D2H = 80FB02H (not 80FB0DH? Let me recalculate carefully)

Let me recalc precisely:

text

```
Physical = Segment × 10H + Offset
80FB0DH = Segment × 10H + BF0D2H
Segment × 10H = 80FB0DH - BF0D2H

80FB0DH - 0BF0D2H:

80FB0D
-0BF0D2
———————
750A3B  (since 80FB0D - 0BF0D2 = 750A3B)

Now Segment = 750A3B ÷ 10H = 750A3 (with remainder B)

Segment = 750A3H

Verification:
750A3 × 10 = 750A30
750A30 + BF0D2 = 80FB02
```

Wait, 80FB02 vs expected 80FB0D → difference of B (11 decimal).
This means the offset BF0D2H is not the correct offset for physical 80FB0DH.

Alternative approach: The question likely expects:
If offset = BF0D2H, then segment = (80FB0D - BF0D2) / 10 = **750A3H**

ELI5:
We know house number (80FB0D) and the room number (BF0D2).
Subtract room number from house number, then divide by 16 to get street number.

4(c) DOS and BIOS Interrupts

BIOS Interrupts (Basic Input/Output System)

Low-level hardware services stored in ROM.

INT Number	Function	Example
INT 10H	Video services	Display character, set mode
INT 13H	Disk services	Read/write sectors
INT 16H	Keyboard services	Read key, check key press

INT Number	Function	Example
INT 17H	Printer services	Send character to printer

DOS Interrupts (INT 21H)

Higher-level operating system services.

AH Value	Function	Example
01H	Read character from STDIN	`MOV AH, 1 INT 21H`
02H	Display character	`MOV DL, 'A' MOV AH, 2 INT 21H`
09H	Display string	`MOV DX, offset msg MOV AH, 9 INT 21H`
4CH	Exit program	`MOV AH, 4CH INT 21H`

Example: "Hello World" using DOS interrupt

assembly

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'Hello, World!$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV DX, OFFSET MSG
    MOV AH, 9           ; DOS display string function
    INT 21H             ; call DOS
    MOV AH, 4CH         ; DOS exit function
    INT 21H
MAIN ENDP
END MAIN
```

ELI5:

- **BIOS** = Factory-installed remote control for TV (hardware buttons)
- **DOS** = Cable TV guide (higher-level channels and programs)
- **INT 21H** = "Hey DOS, do this for me"

SECTION B

Question 5

5(a) Assembler Directives

Definition: Instructions to the **assembler** (not the CPU) that control how the program is assembled.

Common Assembler Directives (MASM/TASM)

Directive	Task	Example
.MODEL	Sets memory model	`.MODEL SMALL`
.STACK	Defines stack size	`.STACK 100H`
.DATA	Start data segment	`.DATA`
.CODE	Start code segment	`.CODE`
DB	Define Byte (8-bit)	`VAR DB 10H`
DW	Define Word (16-bit)	`VAR DW 1234H`
DD	Define Double word (32-bit)	`VAR DD 12345678H`
EQU	Define constant	`MAX EQU 100`
ORG	Set origin address	`ORG 100H` (for .COM files)
END	End of program	`END MAIN`

ELI5:
Directives = Recipe instructions to the chef (assembler), not the dish itself (CPU).
"Preheat oven to 350°" = `.DATA` directive

5(b) Range Problem of Conditional Jump Instructions

The Problem

Conditional jumps in 8086 are **short jumps** (only **-128 to +127 bytes** from next instruction).

text

```
┌──────────────────────────────────────────────────────┐
│  JNZ LABEL  (2 bytes)                                 │
│      │                                                │
│      │  Can only jump within -128 to +127 bytes      │
│      ▼                                                │
│  LABEL: ...                                           │
└──────────────────────────────────────────────────────┘
```

Solution: Reverse Condition + Far Jump

assembly

```
; Problem: Target is too far
        JNZ FAR_LABEL   ; ERROR - not allowed directly

; Solution:
        JZ NEAR_LABEL    ; Jump if ZERO (reverse condition)
        JMP FAR_LABEL    ; Unconditional far jump
NEAR_LABEL:
        ...              ; Rest of code
```

ELI5:
Conditional jump = short step ladder (only reaches nearby)
Solution = If you can't reach, take one small step in opposite direction, then use a long ladder.

5(c) MOV and XCHG Restrictions + Array Element Exchange

MOV Restrictions

Restriction	Example (INVALID)
No memory-to-memory	`MOV [DI], [SI]`
No immediate to segment register	`MOV DS, 1000H`
No segment to segment	`MOV DS, ES`
No CS as destination	`MOV CS, AX`
No immediate to IP	`MOV IP, 100H`

XCHG Restrictions

- Cannot exchange with memory-to-memory (one must be register)
- Cannot exchange segment registers directly
- Cannot exchange with immediate values

Exchange 15th and 30th elements in word array A

assembly

```
.DATA
    A DW 100 DUP(?)      ; Array of 100 words

.CODE
    MOV CX, 14           ; Index of 15th element (0-based)
    MOV BX, 29           ; Index of 30th element (0-based)

    ; Swap using register as temporary
    MOV AX, A[SI]        ; Actually need address calculation
    ; Better method:
    LEA SI, A            ; SI = base address of A
    MOV AX, [SI + 14*2]  ; 15th element (14*2 = 28 bytes
offset)
    MOV DX, [SI + 29*2]  ; 30th element (29*2 = 58 bytes
offset)
    MOV [SI + 14*2], DX
    MOV [SI + 29*2], AX
```

ELI5:
MOV = "Copy from A to B" (can't copy directly between two houses)
XCHG = "Swap contents of two lockers" (needs a third locker for temporary)

Question 6

6(a) Signed Overflow vs Unsigned Overflow

Unsigned Overflow (Carry Flag CF)

When result exceeds maximum value for number of bits (0 to 2^n-1).

Example (8-bit): 255 + 1 = 256 → CF=1

text

```
   1111 1111 (255)
+  0000 0001 (1)
= 1 0000 0000 (256) → CF=1
```

Signed Overflow (Overflow Flag OF)

When result exceeds signed range (-2^{n-1} to $+2^{n-1}-1$).

Example (8-bit): 127 + 1 = -128 (overflow!)

text

```
   0111 1111 (+127)
+  0000 0001 (+1)
=  1000 0000 (-128) → OF=1
```

Comparison Table

Aspect	Unsigned Overflow	Signed Overflow
Flag	CF (Carry)	OF (Overflow)
Range	0 to 2^n-1	-2^{n-1} to $+2^{n-1}-1$
Example (8-bit)	255+1=256 → CF=1	127+1=-128 → OF=1
Detection	Carry out of MSB	Sign changes incorrectly

ELI5:

- Unsigned = Odometer (999 miles + 1 = 000, light comes on)
- Signed = Thermometer (+127°C + 1°C = -128°C, broken!)

6(b) Program Segment Prefix (PSP)

Definition: A 256-byte (100H) block created by DOS before loading a .COM or .EXE program.

PSP Structure

Offset	Size	Content
00H-01H	2 bytes	INT 20H instruction (exit)
02H-03H	2 bytes	Segment of last memory block
05H-09H	5 bytes	FAR call to DOS
10H-11H	2 bytes	Environment segment
2CH-2DH	2 bytes	Environment segment pointer
80H-81H	1 byte	Command line length
81H-FFH	127 bytes	Command line arguments

Memory Layout

text

```
┌──────────────────────────────────┐
│  PSP (256 bytes)                 │ ← CS=DS=ES=SS at start
│  - INT 20H at 0                  │
│  - Command line at 80H           │
├──────────────────────────────────┤
│                                  │
│  Program Code & Data             │
│  (.COM file starts at 100H)      │
│                                  │
└──────────────────────────────────┘
```

ELI5:
PSP = "Welcome packet" DOS gives every program.
Contains: exit pass, command line arguments, environment variables.

6(c) Reverse Bit Pattern of AX without Changing Contents

assembly

```
; Method 1: Using RCL and RCR (rotate through carry)
REVERSE1:
    MOV CX, 16          ; 16 bits to reverse
    MOV DX, 0           ; DX will hold reversed result
    CLC                 ; Clear carry flag
REV1:
    RCL AX, 1           ; Rotate MSB into carry
    RCR DX, 1           ; Rotate carry into DX LSB
    LOOP REV1
    MOV AX, DX          ; AX now reversed
    RET

; Method 2: Using shift and OR (no carry)
REVERSE2:
    MOV CX, 16
    MOV BX, AX          ; BX = original
    XOR DX, DX          ; Clear result
REV2:
    SHR AX, 1           ; Shift LSB into carry
    RCL DX, 1           ; Move carry into DX from left
    LOOP REV2
    MOV AX, DX
    RET
```

Example: AX = 0x1234 (0001 0010 0011 0100)
After reverse: 0010 1100 0100 1000 = 0x2C48

ELI5:
Reversing bits = Mirror image. Like reading binary number backwards.
Method: Take one bit from right, put it into left of new register.

Question 7

7(a) Why TEST and JCXZ Instructions are Used

TEST Instruction

Purpose: Perform AND operation without changing destination (only affects flags).

Use cases:

1. **Check if a bit is set**
   `TEST AL, 00000001B` ; Check LSB
   `JNZ BIT_SET` ; Jump if LSB=1
2. **Check if register is zero**
   `TEST AX, AX` ; AX & AX = AX (sets ZF if AX=0)
   `JZ ZERO` ; Jump if zero

Difference from AND: AND changes destination, TEST does not.

JCXZ Instruction (Jump if CX Zero)

Purpose: Jump if CX register is zero. Used for loop control.

Use cases:

1. **Skip loop if count is zero**
   `JCXZ SKIP_LOOP` ; If CX=0, skip loop
   `LOOP_START: ...` ; Loop body
   `LOOP LOOP_START`
2. **String operations**
   `JCXZ DONE` ; No characters to process

ELI5:

- **TEST** = "Check if light is on without touching it"
- **JCXZ** = "If counter is zero, skip the line"

7(b) Masking – Set MSB and LSB of DL

Masking: Using AND/OR to modify specific bits while preserving others.

Problem: Set most significant bit (bit 7) and least significant bit (bit 0) of DL

DL bits: D7 D6 D5 D4 D3 D2 D1 D0

Solution:

assembly

```
    OR  DL, 10000001B   ; Set bit7 and bit0 to 1
    ; OR with 1 sets bit, OR with 0 preserves
```

General Masking Techniques

Operation	Effect	Example
AND	Clear bits (0 mask)	`AND AL, 0F0H` (clear lower 4 bits)

Operation	Effect	Example
OR	Set bits (1 mask)	`OR AL, 0FH` (set lower 4 bits)
XOR	Toggle bits (1 mask)	`XOR AL, 0FH` (flip lower 4 bits)

ELI5:
Masking = Painting with stencil.

- AND = Stencil that blocks some areas (clears bits)
- OR = Stencil that paints some areas (sets bits)
- XOR = Stencil that flips colors

7(c) Addressing Modes – Indexed Addressing

Addressing Mode: The way CPU calculates the effective address of an operand.

Indexed Addressing Modes

Mode	Formula	Example	Meaning
Direct Indexed	[SI] or [DI]	`MOV AX, [SI]`	AX = memory at DS:SI
Base-Indexed	[BX+SI]	`MOV AX, [BX+SI]`	AX = memory at DS:BX+SI
Indexed with displacement	[SI+disp]	`MOV AX, [SI+10]`	AX = memory at DS:SI+10
Base-Indexed with displacement	[BX+SI+disp]	`MOV AX, [BX+SI+100]`	AX = memory at DS:BX+SI+100

Examples

assembly

```
.DATA
    ARRAY DB 10, 20, 30, 40, 50
.CODE
    LEA SI, ARRAY        ; SI = address of ARRAY
    MOV AL, [SI]         ; AL = 10 (element 0)
    MOV AL, [SI+1]       ; AL = 20 (element 1)
    MOV AL, [SI+2]       ; AL = 30 (element 2)

    ; Access using loop
    MOV CX, 5
    LEA SI, ARRAY
NEXT:
    MOV AL, [SI]         ; AL = current element
    INC SI               ; move to next element
    LOOP NEXT
```

ELI5:
Indexed addressing = "Give me the box at shelf number (base + X)"
SI = your finger pointing to a shelf
[SI+2] = two boxes to the right

Question 8

8(a) What Happens When CALL and RET Execute

CALL Instruction Execution

text

```
Before CALL:            After CALL:
┌──────────────┐        ┌──────────────┐
│    Stack     │        │    Stack     │
├──────────────┤        ├──────────────┤
│              │        │   IP (ret)   │ ← SP (decremented)
│              │        │   CS (ret)   │
└──────────────┘        └──────────────┘

     ↑                       ↑
    SP                      SP
```

Steps:

1. **Push IP** (offset of next instruction after CALL) onto stack
2. **Push CS** (current code segment) onto stack
3. **Load new CS:IP** from the called procedure's address

4. **Jump** to the procedure

RET Instruction Execution

Steps:

1. **Pop CS** from stack (restore code segment)
2. **Pop IP** from stack (restore offset)
3. **Jump** back to calling location

ELI5:
CALL = Leave a sticky note (return address) on the stack, go to friend's house
RET = Read sticky note, come back home

8(b) How CPU Implements Conditional Jump

Implementation Mechanism

Conditional jumps check **flags** in the **Flag Register**.

text

```
┌──────────────────────────────────────────────────────────────┐
│                        Flag Register                         │
│                                                              │
│  ┌──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┐         │
│  │OF│DF│IF│TF│SF│ZF│AF│PF│CF│  │  │  │  │  │  │  │         │
│  └──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┘         │
│   15 14 13 12 11 10 9  8  7  6  5  4  3  2  1  0            │
└──────────────────────────────────────────────────────────────┘
```

Conditional Jump Mnemonics

Mnemonic	Condition	Flags Tested
JE / JZ	Equal / Zero	ZF = 1
JNE / JNZ	Not equal / Not zero	ZF = 0
JG / JNLE	Greater (signed)	ZF=0 and SF=OF
JL / JNGE	Less (signed)	SF ≠ OF
JA / JNBE	Above (unsigned)	CF=0 and ZF=0
JB / JNAE	Below (unsigned)	CF=1

Example

assembly

```
CMP AX, BX        ; Compare AX and BX (sets flags)
; CMP does: AX - BX (subtract, discard result, only flags)

JG  GREATER       ; Jump if AX > BX (signed)
JE  EQUAL         ; Jump if AX = BX
JL  LESS          ; Jump if AX < BX (signed)
```

Hardware implementation:
CPU has a **flag comparator** circuit. For JG:
`Jump = (ZF=0) AND (SF=OF)`
These conditions are checked in a single clock cycle.

ELI5:
Flags = Dashboard warning lights
CMP = Compare two numbers → lights come on (ZF, SF, OF, CF)
Conditional jump = "If OIL light is on, pull over"

8(c) String Operations

(i) Scan a Character from a String (SCAS)

SCASB (Scan String Byte) – compares AL with [ES:DI], then increments/decrements DI.

assembly

```
.DATA
    STRING DB 'Hello World', 0
    SEARCH DB 'o'
.CODE
    LEA DI, STRING      ; ES:DI points to string
    MOV AL, SEARCH      ; Character to search
    MOV CX, 11          ; Length of string
    CLD                 ; Clear direction flag (forward)

SCAN_LOOP:
    SCASB               ; Compare AL with [DI], DI++
    JE  FOUND           ; If equal, jump
    LOOP SCAN_LOOP      ; Continue
    ; Not found
FOUND:
    ; DI points to character after match
```

(ii) Compare a String with Another String (CMPS)

CMPSB – compares [DS:SI] with [ES:DI], updates SI and DI.

assembly

```
.DATA
    STR1 DB 'Computer', 0
    STR2 DB 'Computer', 0
.CODE
    LEA SI, STR1        ; DS:SI → first string
    LEA DI, STR2        ; ES:DI → second string
    MOV CX, 8           ; Compare 8 characters
    CLD                 ; Forward direction
    REPE CMPSB          ; Repeat while equal (ZF=1)

    JNE NOT_EQUAL       ; If ZF=0, strings differ
    ; Strings are equal
NOT_EQUAL:
    ; Strings differ, SI/DI point to mismatch
```

String Instruction Prefixes

Prefix	Meaning
REP	Repeat CX times
REPE / REPZ	Repeat while equal / zero
REPNE / REPNZ	Repeat while not equal / not zero

ELI5:

- **SCAS** = "Find Waldo" – scan string for character
- **CMPS** = "Spot the difference" – compare two strings
- **REP** = "Do this multiple times" – automatic loop

Quick Revision Table – All Key Terms

Term	ELI5
BIU	Waiter bringing food
EU	Mouth chewing food
Segment	Library section
Offset	Shelf number
Multiplexing	Same road for cars & trucks at different times
ALE	Traffic light for address/data
PSP	Welcome packet from DOS
TEST	Check if light is on without touching
JCXZ	If counter=0, skip
Masking	Painting with stencil
Indexed addressing	"Give me box at shelf + X"
CALL	Leave sticky note, go to friend
RET	Read sticky note, come back
SCAS	Find Waldo in string

Term	ELI5
CMPS	Spot difference between strings

Good luck tomorrow! Key points:

- Draw diagrams for 8086 architecture, bus cycles, memory segmentation
- Show flag calculations step by step
- Write actual assembly code for string operations
- Explain with real-world analogies

You will do great! 🚀

image.png

PNG 392.97KB

Here is the complete, exam-oriented solution for **Section A (Questions 1–4)** from your CSE3231 2020 paper.
I have included **diagrams, tables, step-by-step calculations, real-world examples, and ELI5 explanations** as requested.

SECTION A

Question 1

1(a) Memory Banking & Odd/Even Addressing in 8086

What is Memory Banking?

Memory banking is dividing the memory into two independent 8-bit banks:

- **Even bank (lower bank)** – stores low bytes (D0–D7)
- **Odd bank (upper bank)** – stores high bytes (D8–D15)

text

```
┌──────────────────────────────────────────────────────────────┐
│                      1 MB Memory Space                       │
│                                                              │
├──────────────────────────────┬───────────────────────────────┤
│      Even Bank (Low Byte)    │      Odd Bank (High Byte)     │
│                              │                               │
│         D0 - D7              │         D8 - D15              │
│                              │                               │
├──────────────────────────────┼───────────────────────────────┤
│  Address: 00000H (low)       │  Address: 00001H (high)       │
│                              │                               │
│           00002H (low)       │           00003H (high)       │
│                              │                               │
│           00004H (low)       │           00005H (high)       │
│                              │                               │
│           ...                │           ...                 │
│                              │                               │
└──────────────────────────────┴───────────────────────────────┘
```

Odd and Even Addressing

Case 1: Even Address (e.g., 1000H) – Word Access

text

```
One bus cycle:
┌─────────┐      ┌──────────────────────────────┐
│ 8086    │      │ Even: 1000H → Low byte (D0-D7)  │
│         │──────→│ Odd:  1001H → High byte (D8-D15) │
└─────────┘      └──────────────────────────────┘
```

Case 2: Odd Address (e.g., 1001H) – Word Access

text

```
Two bus cycles:
Cycle 1: ┌─────────┐      ┌──────────────────┐
         │ 8086    │──────→│ Odd: 1001H (low)  │
         └─────────┘      └──────────────────┘

Cycle 2: ┌─────────┐      ┌──────────────────┐
         │ 8086    │──────→│ Even: 1002H (high) │
         └─────────┘      └──────────────────┘

CPU then combines: (1002H high) + (1001H low) = word
```

Control signals:

- **BHE** (Bus High Enable) = 0 → odd bank selected
- **A0** = 0 → even bank selected

BHE	A0	Operation
0	0	Whole word (both banks)
0	1	High byte only (odd address)
1	0	Low byte only (even address)
1	1	None (invalid)

ELI5:
Memory banking = Two mailboxes side by side:

- Even address = Left mailbox (low byte)
- Odd address = Right mailbox (high byte)
- Word at even address = Open both at once
- Word at odd address = Open right, then left of next pair

1(b) Difference between 8085 and 8086 Microprocessor

Feature	Intel 8085	Intel 8086
Data lines	8-bit	16-bit
Address lines	16-bit	20-bit
Memory capacity	64 KB (2^{16})	1 MB (2^{20})
Speed (clock)	3–6 MHz	5–10 MHz
Registers	8-bit (can pair)	16-bit
Pipeline	No	Yes (BIU + EU)
Instruction queue	No	6-byte queue
Multiplication	Software only	Hardware MUL instruction
I/O ports	256 (8-bit address)	64 KB (16-bit address)
Power supply	+5V single	+5V single
Pin count	40	40

ELI5:

- **8085** = Bicycle (8 bits at a time, slower, less cargo)
- **8086** = Pickup truck (16 bits at a time, faster, can carry more)

1(c) Steps: Microprocessor Communicating with Peripherals/Memory

text

```
┌──────────────────────────────────────────────────────────────────────────────
│                    Communication Cycle
│
├──────────────────────────────────────────────────────────────────────────────
│
│
│  Step 1: CPU places address on address bus
│
│           ┌──────────┐       ┌──────────────┐
│           │  CPU     │──────→│ Address Bus  │──→
Memory/Peripheral  │
│           └──────────┘       └──────────────┘
│
│
│
│  Step 2: CPU asserts control signals
│
│           ┌──────────┐       ┌──────────────┐
│           │  CPU     │──────→│ RD (read)    │──→
Memory/Peripheral  │
│           │          │       │ WR (write)   │
│           │          │       │ IO/M         │
│           └──────────┘       └──────────────┘
│
│
│
│  Step 3: Data transfer
│
│          FOR READ:  Memory/Peripheral ──→ CPU via data bus
│
│          FOR WRITE: CPU ──→ Memory/Peripheral via data bus
│
│
│
│  Step 4: Wait for acknowledgment (READY signal)
│
│
│
└──────────────────────────────────────────────────────────────────────────────
```

Detailed Steps Table

Step	Action	Control Signals
1	CPU places address on A0–A19	ALE = 1 (address valid)
2	CPU sets IO/M (0=memory, 1=I/O)	IO/M = 0 or 1
3	CPU sets RD or WR	RD=0 (read), WR=0 (write)
4	Memory/peripheral responds	Data on data bus
5	Wait for READY	READY=1 (complete)
6	Latch data (read) or release bus (write)	DT/R, DEN

ELI5:

1. Call out address (like shouting "Apartment 3B!")
2. Say what you want ("I'm reading!" or "I'm writing!")
3. Pass the letter or receive the letter
4. Wait for "Got it!" signal

Question 2

2(a) Multiplexing and Demultiplexing of 8086 Buses

Why Multiplex?

8086 has only 40 pins. To fit 20 address + 16 data, they share pins.

How Multiplexing Works

text

```
Time Slot    Pins AD0-AD15          Pins A16/S3-A19/S6
──────────────────────────────────────────────────────
T1 (Address)  A0-A15 (address)      A16-A19 (address)
T2 (Data)     D0-D15 (data)         Status signals (S3-S6)
```

Demultiplexing Circuit

text

```
┌──────────────────────────────────────┐
│            8086 CPU                  │
│                                      │
│  AD0–AD15 ──┬──┬──┬
```

Address Type	Bus Cycle	Bank Accessed
Odd address	1 cycle	Odd bank only

Word Read

Address Type	Bus Cycles	Action
Even address	1 cycle	Both banks simultaneously
Odd address	2 cycles	Cycle 1: Odd bank (low byte) Cycle 2: Even bank (high byte)

Example: Read word from 1001H (odd)

text

```
Memory:
1000H: 12H (low)   1001H: 34H (low)
1002H: 56H (low)   1003H: 78H (high)

Reading word at 1001H:
Cycle 1: Read 1001H → 34H (becomes low byte of word)
Cycle 2: Read 1002H → 56H (becomes high byte of word)
Result word = 5634H
```

ELI5:
Byte read = picking one apple from one basket
Word at even address = picking two apples from two baskets at once
Word at odd address = pick one apple from right basket, then one from left basket of next pair

Question 3

3(a) Block Diagram of 8086 – EU and BIU

text

```
┌──────────────
```

```
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘
│ │
│
└──────────────────────────────────────────────────────────
│
│                                  │
│                                  │
│                           ┌──────┴──────┐
│
│                           │ 16-bit Data Bus│
│
│                           │ 20-bit Address │
│
│                           │  Bus           │
│
│                           └───────────────┘
│
└──────────────────────────────────────────────────────────
```

Functions of BIU

Function	Description
Fetches instructions	Reads from memory into queue
Generates physical address	Segment × 10H + Offset
Controls buses	Manages address/data/control lines
Prefetch queue	Stores 6 bytes of upcoming instructions

Functions of EU

Function	Description
Decodes instructions	Interprets opcodes from queue
Executes instructions	ALU operations, data movement
Manages registers	General purpose and flags
Controls internal data flow	Directs data between registers

ELI5:

- **BIU** = Waiter (brings food from kitchen)
- **EU** = Mouth and stomach (chews, digests)
- **Queue** = Extra plates on table (pre-fetched food)

3(b) Different Data Registers of 8086

All data registers are **16-bit** and can be accessed as **8-bit** (high/low).

Register	High Byte	Low Byte	Special Use
AX (Accumulator)	AH	AL	Arithmetic, I/O operations
BX (Base)	BH	BL	Base pointer for memory access
CX (Counter)	CH	CL	Loop counter, shift count
DX (Data)	DH	DL	I/O port address, multiply/divide

Register Diagram

text

```
┌──────────────────────────────────────────────────────────────┐
│                          AX                                  │
│                                                              │
│              ┌──────────────┬──────────────┐                 │
│              │      AH      │      AL      │                 │
│              └──────────────┴──────────────┘                 │
│                                                              │
│               15-8 bits       7-0 bits                       │
├──────────────────────────────────────────────────────────────┤
│                          BX                                  │
│                                                              │
│              ┌──────────────┬──────────────┐                 │
│              │      BH      │      BL      │                 │
│              └──────────────┴──────────────┘                 │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                          CX                                  │
│                                                              │
│              ┌──────────────┬──────────────┐                 │
│              │      CH      │      CL      │                 │
│              └──────────────┴──────────────┘                 │
│                                                              │
├──────────────────────────────────────────────────────────────┤
│                          DX                                  │
│                                                              │
│              ┌──────────────┬──────────────┐                 │
│              │      DH      │      DL      │                 │
│              └──────────────┴──────────────┘                 │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

ELI5:
AX = Main workbench (most operations here)
BX = Pointer finger (points to memory)
CX = Counter (counts loops)
DX = Special assistant (helps with I/O and math)

3(c) Flags of 8086 Microprocessor

Flag Register is a 16-bit register where each bit indicates a condition.

Flag Register Layout

text

```
  15  14  13  12  11  10   9   8   7   6   5   4   3   2   1
0
┌───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬───┬
│   │   │   │   │ OF │ DF │ IF │ TF │ SF │ ZF │   │ AF │   │
PF │   │ CF │
└───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴───┴
```

Status Flags (Arithmetic/Logic)

Flag	Name	Set When	Example
CF	Carry	Unsigned overflow	255+1=256
PF	Parity	Low 8 bits have even number of 1's	0x03 (00000011 → 2 ones → PF=1)
AF	Aux Carry	Carry from bit 3 to bit 4 (BCD)	0x0F+0x01=0x10
ZF	Zero	Result is zero	5-5=0
SF	Sign	MSB of result = 1 (negative)	0x80 (10000000)
OF	Overflow	Signed overflow	127+1=-128

Control Flags

Flag	Name	Purpose
IF	Interrupt	Enable/disable interrupts
DF	Direction	String direction (0=up, 1=down)

Flag	Name	Purpose
TF	Trap	Single-step debugging

ELI5:
Flags = Dashboard warning lights:

- CF = "Check engine" (overflow)
- ZF = "Empty tank" (zero result)
- SF = "Negative battery" (negative number)
- OF = "Speed too high" (signed overflow)

Question 4

4(a) Memory Segmentation Concept of 8086

Segmentation = Dividing 1MB memory into 64KB logical blocks called **segments**.

Why Segmentation?

Reason	Explanation
20-bit address from 16-bit registers	Segment × 10H + offset = 20 bits
Program relocation	Code can be loaded anywhere
Multiple programs	Each program has its own segments
Code/Data separation	Different segments for different purposes

Memory Segmentation Diagram

text

```
Physical Memory (1MB)                Logical View
────────────────────                 ┌─────────────────┐
00000 ┌─────────────────┐            │  Code Segment   │ ← CS:IP
      │                 │            │  (64KB max)     │
      │   Code Segment  │            └─────────────────┘
      │   (64KB)        │            ┌─────────────────┐
10000 ┼─────────────────┼─┐          │  Data Segment   │ ←
DS:BX
      │                 │ │          │  (64KB max)     │
      │   Data Segment  │ │          └─────────────────┘
      │   (64KB)        │ │          ┌─────────────────┐
20000 ┼─────────────────┼─┘          │  Stack Segment  │ ←
SS:SP
      │                 │            │  (64KB max)     │
      │   Stack Segment │            └─────────────────┘
      │   (64KB)        │            ┌─────────────────┐
30000 ┼─────────────────┼─┐          │  Extra Segment  │ ←
ES:DI
      │                 │ │          │  (64KB max)     │
      │   Extra Segment │ │          └─────────────────┘
      │   (64KB)        │ │
40000 ┼─────────────────┼─┘
      │                 │
      │   Other         │
FFFFF └─────────────────┘
```

ELI5:
Memory = Library
Segments = Sections (Fiction, Non-fiction, Reference)
Offset = Shelf number within section

4(b) Role of Segment and Offset Registers

Segment Registers

Register	Role	Default Offset	ELI5
CS	Code Segment (instructions)	IP	Reading room
DS	Data Segment (variables)	BX, SI, DI	Book stacks
SS	Stack Segment (stack)	SP, BP	Break room

Register	Role	Default Offset	ELI5
ES	Extra Segment (string dest)	DI	Copy room

Offset Registers

Register	Role	Default Segment	ELI5
IP	Instruction Pointer (next instruction)	CS	Bookmark in reading room
SP	Stack Pointer (top of stack)	SS	Top plate in stack
BP	Base Pointer (stack frame)	SS	Reference point in stack
SI	Source Index (string source)	DS	Pointer to source data
DI	Destination Index (string dest)	ES	Pointer to destination
BX	Base register (data pointer)	DS	General pointer

How They Work Together

text

```
Physical Address = (Segment × 10H) + Offset

Example: CS = 0A32H, IP = 0028H
Physical = 0A320H + 0028H = 0A348H
```

ELI5:
Segment = Street number × 16
Offset = House number
Together = Complete address

4(c) Segment:Offset from Physical Address

Given: Physical address = **1256AH**

For Segment = 1256H

text

```
Formula: Physical = (Segment × 10H) + Offset
1256AH = (1256H × 10H) + Offset
1256AH = 12560H + Offset
Offset = 1256AH - 12560H = 000AH

Answer: 1256H:000AH
```

For Segment = 1240H

text

```
1256AH = (1240H × 10H) + Offset
1256AH = 12400H + Offset
Offset = 1256AH - 12400H = 016AH

Answer: 1240H:016AH
```

Verification

Segment	Offset	Calculation	Result
1256H	000AH	12560H + 000AH = 1256AH ✓	
1240H	016AH	12400H + 016AH = 1256AH ✓	

ELI5:
Same house (1256AH) can be described from different streets:

- From street 1256H: just 10 houses down
- From street 1240H: 362 houses down (016AH = 362 decimal)

Quick Revision Table – All Key Terms

Term	ELI5
Memory banking	Two mailboxes side by side
BHE	"Right mailbox key"
Multiplexing	Same road for cars & trucks at different times

Term	ELI5
Demultiplexing	Traffic cop separates traffic
BIOS	Factory remote (direct hardware)
DOS	Cable box (extra features)
BIU	Waiter bringing food
EU	Mouth chewing food
AX, BX, CX, DX	Workbench, pointer, counter, assistant
Flags	Dashboard warning lights
Segmentation	Library sections
Segment:Offset	Street × 16 + house number

Good luck tomorrow! Key points to remember:

- Draw the 8086 architecture diagram (BIU + EU)
- Show memory banking with BHE and A0 table
- Calculate physical addresses step by step
- Use ELI5 analogies in your answers for better marks

You've got this! 🚀

image.png

PNG 383.34KB

Here is the complete, exam-oriented solution for **Section B (Questions 5–8)** from your CSE3231 2020 paper.
I have included **assembly code, diagrams, tables, step-by-step explanations, real-world examples, and ELI5 summaries** as requested.

SECTION B

Question 5

5(a) Assembly Code: Read Character, Display if 'y' or 'Y'

assembly

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Enter a character: $'
    MSG2 DB 'You entered: $'
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    ; Initialize data segment
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, MSG1
    MOV AH, 9          ; DOS display string function
    INT 21H

    ; Read character
    MOV AH, 1          ; DOS read character function
    INT 21H            ; AL = input character

    ; Store character
    MOV BL, AL         ; Save in BL

    ; Check if 'y' or 'Y'
    CMP AL, 'y'        ; Compare with lowercase y
    JE  DISPLAY        ; If equal, display
    CMP AL, 'Y'        ; Compare with uppercase Y
    JE  DISPLAY        ; If equal, display

    ; Not 'y' or 'Y' → terminate
    JMP TERMINATE

DISPLAY:
    ; Print newline
    LEA DX, NEWLINE
    MOV AH, 9
    INT 21H

    ; Display message
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; Display the character
    MOV DL, BL
    MOV AH, 2          ; DOS display character function
    INT 21H

TERMINATE:
    ; Exit program
    MOV AH, 4CH
    INT 21H

MAIN ENDP
END MAIN
```

Flowchart

text

```
        ┌─────────────┐
        │   START     │
        └─────────────┘
              ▼
        ┌─────────────┐
        │ Display     │
        │ prompt      │
        └─────────────┘
              ▼
        ┌─────────────┐
        │ Read char   │
        │ into AL     │
        └─────────────┘
              ▼
        ┌─────────────┐
        │ AL = 'y'?   │──────No──────┐
        └─────────────┘              │
             Yes                     │
              │                      ▼
              │              ┌─────────────┐
              │              │ AL = 'Y'?   │──────No──────┐
              │              └─────────────┘              │
              │                   Yes                     │
              │                    │                      │
              ▼                    ▼                      ▼
        ┌─────────────┐  ┌─────────────┐  ┌─────────────┐
        │ Display     │  │ Display     │  │ Terminate   │
        │ character   │  │ character   │  │ program     │
        └─────────────┘  └─────────────┘  └─────────────┘
```

ELI5:

This program is like a security guard:

"Say the password. If you say 'yes' or 'YES', I'll let you in. Otherwise, goodbye!"

5(b) Using AND, OR, XOR to Clear, Set, Complement Bits

AND – Clear Specific Bits (Masking)

Formula: `AND destination, mask` (bit with 0 → clears; with 1 → preserves)

assembly

```
; Clear bits 3 and 1 (0-indexed) of AL
; Bit positions: 7 6 5 4 3 2 1 0
; Want to clear bits 3 and 1 → mask = 11110101B = 0xF5

MOV AL, 0xFF          ; AL = 1111 1111
AND AL, 0xF5          ; AL = 1111 0101 (bits 3 and 1 cleared)

; Result: bits 3 and 1 become 0
```

Diagram:

text

```
Before AND:  1  1  1  1  1  1  1  1
Mask:        1  1  1  1  0  1  0  1
After AND:   1  1  1  1  0  1  0  1
                    ↑     ↑
                cleared cleared
```

OR – Set Specific Bits

Formula: `OR destination, mask` (bit with 1 → sets; with 0 → preserves)

assembly

```
; Set bits 5 and 2 of AL
; Mask = 00100100B = 0x24

MOV AL, 0x00          ; AL = 0000 0000
OR  AL, 0x24          ; AL = 0010 0100 (bits 5 and 2 set)

; Result: bits 5 and 2 become 1
```

Diagram:

text

```
Before OR:   0  0  0  0  0  0  0  0
Mask:        0  0  1  0  0  1  0  0
After OR:    0  0  1  0  0  1  0  0
                  ↑        ↑
                 set      set
```

XOR – Complement (Toggle) Specific Bits

Formula: `XOR destination, mask` (bit with 1 → toggles; with 0 → preserves)

assembly

```
; Toggle bits 7 and 0 of AL
; Mask = 10000001B = 0x81

MOV AL, 0x00          ; AL = 0000 0000
XOR AL, 0x81          ; AL = 1000 0001 (bits 7 and 0 set)

XOR AL, 0x81          ; Toggle again → AL = 0000 0000 (back)
```

Diagram:

text

```
First XOR:   0  0  0  0  0  0  0  0
Mask:        1  0  0  0  0  0  0  1
Result:      1  0  0  0  0  0  0  1  (bits toggled ON)

Second XOR:  1  0  0  0  0  0  0  1
Mask:        1  0  0  0  0  0  0  1
Result:      0  0  0  0  0  0  0  0  (bits toggled OFF)
```

Summary Table

Operation	Effect	Example (bit 3)
AND with 0	Clear	`AND AL, 11110111B`
AND with 1	Preserve	`AND AL, 11111111B`
OR with 0	Preserve	`OR AL, 00000000B`
OR with 1	Set	`OR AL, 00001000B`
XOR with 0	Preserve	`XOR AL, 00000000B`

Operation	Effect	Example (bit 3)
XOR with 1	Toggle	`XOR AL, 00001000B`

ELI5:

- **AND** = Paint roller with cutouts (clears where cutout is)
- **OR** = Stamp pad (sets where stamp hits)
- **XOR** = Light switch (flips where finger touches)

5(c) Why `MOV AH, 4CH` and `INT 21H`?

These two instructions **terminate the program** and return control to DOS.

Breakdown

Instruction	What it does
`MOV AH, 4CH`	Load 4CH (76 decimal) into AH – DOS function number for "exit program"
`INT 21H`	Call DOS interrupt handler

Why needed?

Reason	Explanation
Clean exit	Returns memory to DOS, closes files
Prevents crash	Without it, CPU would execute garbage after program
Return code	Can pass exit code in AL (0 = success)
Standard practice	Required for all .EXE programs

Example with return code

assembly

```
MOV AH, 4CH
MOV AL, 0         ; Return code 0 (success)
INT 21H
```

ELI5:
`MOV AH, 4CH` = "Tell the operator I want to go home"
`INT 21H` = "Call the operator"
Together = "DOS, I'm done. Please clean my desk."

Question 6

6(a) Signed Overflow vs Unsigned Overflow

Unsigned Overflow (Carry Flag CF)

Occurs when result exceeds maximum value for number of bits.

Example (8-bit): 255 + 1 = 256

text

```
   1111 1111  (255)
+  0000 0001  (1)
= 1 0000 0000  (256) → CF = 1
```

Signed Overflow (Overflow Flag OF)

Occurs when result exceeds signed range (-128 to +127 for 8-bit).

Example (8-bit): 127 + 1 = -128 (overflow!)

text

```
   0111 1111  (+127)
+  0000 0001  (+1)
=  1000 0000  (-128) → OF = 1
```

Comparison Table

Aspect	Unsigned Overflow	Signed Overflow
Flag	CF (Carry)	OF (Overflow)
Range	0 to 2^n-1	-2^{n-1} to $+2^{n-1}$-1
8-bit range	0 to 255	-128 to +127
Example	255+1=256 → CF=1	127+1=-128 → OF=1

Aspect	Unsigned Overflow	Signed Overflow
Detection	Carry out of MSB	Sign change inconsistency

Detection Rules

Operation	Unsigned	Signed
Addition	CF = 1	OF = 1 if signs same but result sign different
Subtraction	CF = 1 if borrow	OF = 1 if signs different but result sign same

ELI5:

- **Unsigned** = Odometer (999 miles + 1 = 000, light comes on)
- **Signed** = Thermometer (+127°C + 1°C = -128°C → broken!)

6(b) SAL and SAR Instructions of 8086

SAL (Shift Arithmetic Left) – Same as SHL

Shifts bits left, fills LSB with 0, MSB goes to CF.

text

```
Before: 1 0 1 1 0 0 1 0
SAL 1:  0 1 1 0 0 1 0 0  (CF = 1)
```

Effect: Multiplies by 2 (signed or unsigned)

SAR (Shift Arithmetic Right)

Shifts bits right, **preserves sign bit** (MSB), LSB goes to CF.

text

```
Positive number (MSB=0):
Before: 0 1 0 1 1 0 0 1
SAR 2:  0 0 0 1 0 1 1 0  (CF from LSB)

Negative number (MSB=1):
Before: 1 0 1 1 0 0 1 0
SAR 2:  1 1 1 0 1 1 0 0  (sign preserved)
```

Effect: Divides by 2 (signed division, rounds toward -∞)

Comparison: SAR vs SHR

Instruction	Fills with	Use
SHR (Logical)	0	Unsigned divide by 2
SAR (Arithmetic)	Sign bit	Signed divide by 2

ELI5:

- **SAL** = Slide left, add zero (multiply by 2)
- **SAR** = Slide right, keep the sign (signed divide by 2)

6(c) Reverse Bit Pattern of BX without Changing Contents

assembly

```
; Method 1: Using RCL and RCR (rotate through carry)
REVERSE_BX1:
    MOV CX, 16          ; 16 bits to reverse
    MOV AX, 0           ; AX will hold reversed result
    MOV BX, BX          ; BX original (preserved? Wait, this
destroys BX)
    ; Actually, we need to preserve BX
    PUSH BX             ; Save original
    MOV AX, BX          ; Copy to AX
    XOR BX, BX          ; Clear BX for result
    MOV CX, 16
REV1:
    SHR AX, 1           ; Shift LSB into CF
    RCL BX, 1           ; Rotate CF into BX from left
    LOOP REV1
    ; BX now has reversed bits
    POP BX              ; Restore original BX (or swap)
    ; To preserve original BX and put result in another
register:
    ; Result is in BX, but original lost. Let's fix:

; Method 2: Preserve BX, result in AX
REVERSE_BX2:
    MOV CX, 16
    MOV AX, BX          ; AX = original
    XOR DX, DX          ; DX will hold result
REV2:
    SHR AX, 1           ; Shift LSB into CF
    RCL DX, 1           ; Move CF into DX from right
    LOOP REV2
    MOV AX, DX          ; AX = reversed
    ; BX still contains original
    RET

; Method 3: Using XOR swap in place (destroys BX, result in
BX)
REVERSE_BX3:
    MOV CX, 8           ; 8 swaps for 16 bits
    MOV AX, BX
REV3:
    SHR AX, 1
    RCL BX, 1
    LOOP REV3
    RET
```

Example: BX = 0x1234 (0001 0010 0011 0100)
After reverse: 0010 1100 0100 1000 = 0x2C48

ELI5:
Reversing bits = Mirror image. Like reading binary number backwards.
Method: Take one bit from right, put it into left of new register.

Question 7

7(a) Masking – Set MSB and LSB of BL

Masking: Using AND/OR to modify specific bits while preserving others.

Problem: Set bit 7 (MSB) and bit 0 (LSB) of BL

BL bits: b7 b6 b5 b4 b3 b2 b1 b0

Solution:

assembly

```
    OR  BL, 10000001B   ; Set bit7 and bit0 to 1
    ; OR with 1 sets bit, OR with 0 preserves
```

Explanation

Bit position	7	6	5	4	3	2	1	0
Original BL	x	x	x	x	x	x	x	x
Mask	1	0	0	0	0	0	0	1
Result	1	x	x	x	x	x	x	1

ELI5:
Masking = Painting with stencil.
The mask `10000001B` is a stencil that only paints the first and last spots.

7(b) Ways to Clear the Content of a Register

Method	Instruction	Example (AX)
1. MOV immediate	`MOV AX, 0`	`MOV AX, 0`
2. XOR with itself	`XOR AX, AX`	Fastest, 2 bytes
3. SUB from itself	`SUB AX, AX`	Also works

Method	Instruction	Example (AX)
4. AND with zero	`AND AX, 0`	Works but slower
5. Shift out	`SHR AX, 16`	Only for 16-bit
6. POP from stack	`POP AX`	If zero on stack

Best Practice: XOR AX, AX

assembly

```
XOR AX, AX      ; Clears AX, fastest (2 bytes, 2 clocks)
```

Why XOR is best:

- Shortest encoding (2 bytes vs 3 for MOV AX,0)
- Fastest execution
- No memory access
- Works for any register

ELI5:
Clearing a register = Erasing a whiteboard.
`XOR AX, AX` = Erase by writing same thing on itself (cancels out)

7(c) Why TEST and JCXZ Instructions are Used

TEST Instruction

Purpose: Perform AND without changing destination (only affects flags).

Use cases:

1. **Check if a bit is set**

assembly

```
TEST AL, 00000001B   ; Check LSB
JNZ BIT_SET          ; Jump if LSB = 1
```

2. **Check if register is zero**

assembly

```
TEST AX, AX          ; AX & AX = AX (sets ZF if AX=0)
JZ ZERO              ; Jump if zero
```

Difference from AND: AND changes destination, TEST does not.

JCXZ Instruction (Jump if CX Zero)

Purpose: Jump if CX register is zero.

Use cases:

1. **Skip loop if count is zero**

assembly

```
    JCXZ SKIP_LOOP     ; If CX=0, skip loop
LOOP_START:
    ...                ; Loop body
    LOOP LOOP_START
SKIP_LOOP:
```

2. **String operations**

assembly

```
    JCXZ DONE          ; No characters to process
```

ELI5:

- **TEST** = "Check if light is on without touching it"
- **JCXZ** = "If counter is zero, skip the line"

Question 8

8(a) Flag – Signed and Unsigned Overflow Effects

Flag Register is a 16-bit register where each bit indicates a condition from the last arithmetic/logic operation.

How Flags are Affected

Condition	Unsigned (CF)	Signed (OF)
Addition	CF = 1 if result > 2^n-1	OF = 1 if same signs → opposite sign result
Subtraction	CF = 1 if borrow needed	OF = 1 if different signs → opposite sign result
Multiplication	CF/OF = 1 if result > register size	Same
Division	Undefined if overflow	Undefined

Example: 8-bit Addition

Operation	Unsigned (CF)	Signed (OF)
255 + 1 = 256	CF = 1 (overflow)	Not applicable
127 + 1 = 128	CF = 0 (within 0-255)	OF = 1 (127+1=-128)
128 + 128 = 256	CF = 1 (256>255)	OF = 1 (negative+negative=positive?)

ELI5:
Flag register = Dashboard with two warning lights:

- CF = "Check engine" (unsigned overflow)
- OF = "Speed too high" (signed overflow)

8(b) Flags After ADD AX, BX

Given: AX = FFFFH, BX = FFFFH, Instruction: `ADD AX, BX`

Calculation:

text

```
   FFFF
+  FFFF
= 1 FFFE  (1 is carry out)

Binary: 1111 1111 1111 1111
      + 1111 1111 1111 1111
      = 1 1111 1111 1111 1110
        ↑
     Carry (CF=1)
```

Result in AX = FFFEH (16-bit result)

Flag Status Table

Flag	Value	Calculation
CF (Carry)	1	Carry out of MSB
PF (Parity)	0	FFFE = 1111 1111 1111 1110 → 15 ones (odd)
AF (Aux Carry)	1	Carry from bit 3 to bit 4
ZF (Zero)	0	Result ≠ 0
SF (Sign)	1	MSB of result = 1 (negative)
OF (Overflow)	0	Both positive? Actually FFFF is -1 signed. Let's check: -1 + -1 = -2 (no overflow)

ELI5:
Two large negative numbers (-1 each) added = -2. No overflow.
But the addition produced a carry (CF=1) because unsigned value exceeded 65535.

8(c) Program Segment Prefix (PSP)

Definition: A 256-byte (100H) block created by DOS before loading a program.

PSP Structure

Offset	Size	Content
00H-01H	2 bytes	INT 20H instruction (exit)

Offset	Size	Content
02H-03H	2 bytes	Segment of last memory block
05H-09H	5 bytes	FAR call to DOS
10H-11H	2 bytes	Environment segment
2CH-2DH	2 bytes	Environment segment pointer
80H	1 byte	Command line length
81H-FFH	127 bytes	Command line arguments (ASCIIZ)

Memory Layout

text

```
Address (Hex)    Content
_____________________________
0000             INT 20H (CD 20)
0002             Segment of last block
...
0080             Command line length (e.g., 05)
0081             'H' 'E' 'L' 'L' 'O' (arguments)
...
00FF             End of PSP
0100             Program starts here (.COM file)
```

Uses of PSP:

1. Program exit (`INT 20H` at offset 0)
2. Access command line arguments
3. Get environment variables
4. DOS function dispatch

ELI5:
PSP = "Welcome packet" DOS gives every program.
Contains: exit pass, command line arguments, environment settings.

8(d) Code to Load Flags Register to AX

assembly

```
; Method 1: PUSHF and POP
PUSHF               ; Push flags onto stack
POP AX              ; Pop flags into AX

; Method 2: LAHF (Load AH from Flags) - only low byte
LAHF                ; Load AH = (SF, ZF, AF, PF, CF)
; Note: LAHF only gets low 8 bits of flags

; Method 3: PUSHF with memory
PUSHF
POP AX              ; Same as method 1

; Example: Display flags
DISPLAY_FLAGS:
    PUSHF
    POP AX          ; AX now contains flag register
    ; Now AX bits: 15-12 unused, 11=OF, 10=DF, 9=IF, 8=TF,
    ; 7=SF, 6=ZF, 4=AF, 2=PF, 0=CF
```

Flag Bit Positions in AX after PUSHF/POP

Bit	Flag	Bit	Flag
0	CF (Carry)	8	TF (Trap)
1	Reserved	9	IF (Interrupt)
2	PF (Parity)	10	DF (Direction)
3	Reserved	11	OF (Overflow)
4	AF (Aux Carry)	12-15	Reserved
5	Reserved		
6	ZF (Zero)		
7	SF (Sign)		

ELI5:
`PUSHF` = Take a snapshot of all dashboard lights (flags)
`POP AX` = Put that snapshot into a folder (AX register)

Quick Revision Table – Section B

Term	ELI5
AND	Paint roller with cutouts (clears bits)
OR	Stamp pad (sets bits)
XOR	Light switch (toggles bits)
MOV AH,4CH + INT 21H	"DOS, I'm done. Clean my desk."
SAL	Slide left, add zero (×2)
SAR	Slide right, keep sign (signed ÷2)
TEST	Check light without touching
JCXZ	"If counter=0, skip"
PSP	Welcome packet from DOS
PUSHF/POP AX	Snapshot dashboard lights

Good luck tomorrow! Key points to remember:

- Write complete assembly code with proper syntax
- Show AND/OR/XOR with binary examples
- Explain CF vs OF with numeric examples
- Use PUSHF/POP to access flags

You've got this! 🚀

image.png

PNG 398.36KB

Here is the complete, exam-oriented solution for **Section A (Questions 1–4)** from your CSE3231 2019 paper.
I have included **diagrams, tables, step-by-step calculations, real-world examples, and ELI5 explanations** as requested.

SECTION A

Question 1

1(a) 64-bit Processor vs 32-bit Processor

What is a 64-bit Processor?

A **64-bit processor** is a CPU that can:

- Handle **64 bits of data** in one operation
- Has **64-bit data bus** (transfer 8 bytes at a time)
- Has **64-bit general purpose registers**
- Can theoretically address **2^{64} bytes** of memory (16 exabytes)

Differences: 64-bit vs 32-bit Processor

Feature	32-bit Processor	64-bit Processor
Data bus width	32 bits (4 bytes)	64 bits (8 bytes)
Address bus width	32 bits	36-64 bits (typically 48 bits physical)
Maximum memory	4 GB (2^{32})	16 EB (2^{64}) theoretically
Registers	32-bit general purpose	64-bit general purpose
Performance	Lower	Higher (more data per cycle)
Examples	Intel Pentium, 80386	Intel Core i7, AMD Ryzen, ARM Cortex-A
Software	32-bit OS/apps	Can run 32-bit and 64-bit apps

Diagram: 32-bit vs 64-bit Data Transfer

text

```
32-bit Processor (4 bytes at a time)
┌──────────┐      ┌──────────────────────────────────┐
│   CPU    │────→│ Byte0 | Byte1 | Byte2 | Byte3      │
└──────────┘      └──────────────────────────────────┘
                 4 bytes per transfer

64-bit Processor (8 bytes at a time)
┌─────────┐
┌──────────────────────────────────────────────────────────┐
│   CPU   │────→| B0 | B1 | B2 | B3 | B4 | B5 | B6 | B7
│
└─────────┘
└──────────────────────────────────────────────────────────┘
                 8 bytes per transfer (twice as much!)
```

ELI5:

- **32-bit** = 4-lane highway (4 cars at a time)
- **64-bit** = 8-lane highway (8 cars at a time) – twice the traffic!

1(b) Different Categories of Functions Performed by a Microprocessor

Category	Functions	ELI5
1. Data Transfer	MOV, PUSH, POP, IN, OUT, XCHG	Moving boxes between rooms
2. Arithmetic	ADD, SUB, MUL, DIV, INC, DEC	Math operations (add, subtract, multiply)
3. Logical	AND, OR, XOR, NOT, TEST	Logic operations (true/false)
4. Shift/Rotate	SHL, SHR, ROL, ROR, SAR, SAL	Sliding bits left/right
5. Control Transfer	JMP, CALL, RET, LOOP, JCXZ	Changing program flow (branching)
6. String Operations	MOVS, CMPS, SCAS, LODS, STOS	Moving/Comparing blocks of data
7. Flag Control	STC, CLC, STD, CLD, CLI, STI	Setting/resetting status lights
8. Processor Control	HLT, WAIT, NOP, LOCK	Stopping/waiting/syncing

Diagram: Function Categories

text

```
                    ┌──────────────────────┐
                    │   MICROPROCESSOR     │
                    │   FUNCTIONS          │
                    └──────────────────────┘
                              |
        ┌──────────┬──────────┬──────────┬──────────┬──────────┐
        ▼          ▼          ▼          ▼          ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
┌──────────────┐ ┌──────────────┐
│   Data       | | Arithmetic   | | Logical      | |   Shift/
| |  Control   |
|  Transfer    | |              | |              | |   Rotate
| |  Transfer  |
└──────────────┘ └──────────────┘ └──────────────┘
└──────────────┘ └──────────────┘
       |              |              |              |
|
       ▼              ▼              ▼              ▼
▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
┌──────────────┐ ┌──────────────┐
│   String     | |   Flag       | | Processor    | |
| |            |
|  Operations  | |   Control    | |   Control    | |
| |            |
└──────────────┘ └──────────────┘ └──────────────┘
└──────────────┘ └──────────────┘
```

1(c) What is an Interrupt?

Definition: An interrupt is a signal that temporarily suspends the normal execution of a program to execute a special routine (Interrupt Service Routine – ISR) in response to an event.

Types of Interrupts

Type	Source	Example
Hardware	External device	Keyboard press, timer, mouse movement
Software	INT instruction	`INT 21H` (DOS service call)
Internal	CPU exception	Division by zero, overflow

Interrupt Process Flowchart

text

```
┌──────────────────────────────────────────────────────────────────────────────
│                    INTERRUPT PROCESS
│
├──────────────────────────────────────────────────────────────────────────────
│
│
│   ┌──────────┐      ┌──────────┐      ┌──────────┐
┌──────────┐  │
│   │ Normal   │──────→│Interrupt│──────→│ Save     │──────→│ Execute
│  │
│   │Program   │      │ Occurs   │      │ Context  │      │   ISR
│  │
│   │Running   │      │          │      │ (CS,IP,  │      │
│  │
│   └──────────┘      └──────────┘      │  Flags)  │
└──────┬───────┘  │
│                                       └──────────┘           │
│
│                                                              ▼
│
│   ┌──────────┐      ┌──────────┐      ┌──────────┐
┌──────────┐  │
│   │ Resume   │←──────│ Restore │←──────│   IRET   │←──────│ Finish
│  │
│   │ Program  │      │ Context  │      │          │      │  ISR
│  │
│   └──────────┘      └──────────┘      └──────────┘
└──────────┘  │
│
│
└──────────────────────────────────────────────────────────────────────────────
```

ELI5:
Interrupt = Phone ringing while you're reading.
You bookmark your page (save context), answer the call (execute ISR), then return to reading (resume program).

Question 2

2(a) Multiplexing and Demultiplexing of 8086 Address/Data Buses

Why Multiplex?

8086 has 40 pins. To fit 20 address + 16 data, they share pins AD0–AD15.

How Multiplexing Works

Time Slot	AD0–AD15 Pins	A16/S3–A19/S6 Pins
T1 (Address)	A0–A15 (address)	A16–A19 (address)
T2, T3, T4 (Data)	D0–D15 (data)	Status (S3–S6)

Demultiplexing Circuit

text

```
┌──────────────────────────────────────┐
│           8086 CPU                   │
│                                      │
│  AD0–AD15 ──┬──┬──┬
```

ELI5:
Even address = Open both mailboxes at once
Odd address = Open right mailbox, then left mailbox of next pair

2(c) Status Flags After ADD AX, BX

Given: AX = FFFFH, BX = FFFFH, Instruction: `ADD AX, BX`

Calculation:

text

```
   FFFF
+  FFFF
= 1 FFFE  (1 is carry out)

Binary: 1111 1111 1111 1111
      + 1111 1111 1111 1111
      = 1 1111 1111 1111 1110
        ↑
     Carry (CF=1)
```

Result in AX = FFFEH

Flag Status

Flag	Value	Explanation
CF (Carry)	1	Carry out of MSB (unsigned overflow)
PF (Parity)	0	FFFE = 1111 1111 1111 1110 → 15 ones (odd)
AF (Aux Carry)	1	Carry from bit 3 to bit 4
ZF (Zero)	0	Result ≠ 0
SF (Sign)	1	MSB of result = 1 (negative)
OF (Overflow)	0	Signed: -1 + -1 = -2 (no overflow)

ELI5:
Two negative numbers (-1 each) added = -2. No signed overflow.
But unsigned addition produced a carry because 65535+65535=131070 > 65535.

Question 3

3(a) Block Diagram of Intel 8086 – EU and BIU

text

```
┌──────────────
```

- **EU** = Mouth and stomach (chews, digests)
- **Queue** = Extra plates on table

3(b) CS and DS – Why and How Used

CS (Code Segment)

- Points to the segment containing **program instructions**
- Used with **IP (Instruction Pointer)** to fetch code
- **Cannot be modified directly** (for security)

DS (Data Segment)

- Points to the segment containing **program data** (variables)
- Used with BX, SI, DI, or immediate offsets
- **Can be modified** to access different data areas

How They Are Used in Assembly

assembly

```
.DATA
    MSG DB 'Hello$'     ; Data in DS segment

.CODE
MAIN PROC
    MOV AX, @DATA       ; Get address of data segment
    MOV DS, AX          ; Load into DS register

    LEA DX, MSG         ; DS:DX points to string
    MOV AH, 9
    INT 21H             ; Display string from DS:DX
```

Why CS and DS are Needed

Reason	Explanation
Code/Data separation	Instructions separate from variables
Multiple programs	Each program has its own CS and DS
Protection	Cannot accidentally execute data
Relocation	Program can load anywhere in memory

ELI5:

- **CS** = "Reading room" where instructions live
- **DS** = "Book stacks" where data lives
- You tell the CPU: "Look in reading room for next step, look in book stacks for numbers"

3(c) Segment:Offset from Physical Address

Given: Physical address = **1256AH**

For Segment = 1256H

text

```
Formula: Physical = (Segment × 10H) + Offset
1256AH = (1256H × 10H) + Offset
1256AH = 12560H + Offset
Offset = 1256AH - 12560H = 000AH

Answer: 1256H:000AH
```

For Segment = 1240H

text

```
1256AH = (1240H × 10H) + Offset
1256AH = 12400H + Offset
Offset = 1256AH - 12400H = 016AH

Answer: 1240H:016AH
```

Verification

Segment	Offset	Calculation	Result
1256H	000AH	12560H + 000AH = 1256AH ✓	

Segment	Offset	Calculation	Result
1240H	016AH	12400H + 016AH = 1256AH ✓	

ELI5:
Same house (1256AH) can be described from different streets:

- From street 1256H: just 10 houses down
- From street 1240H: 362 houses down (016AH = 362 decimal)

Question 4

4(a) Assembler Directives and Pseudo Opcodes

Definition: Instructions to the **assembler** (not the CPU) that control how the program is assembled.

Common Assembler Directives (MASM/TASM)

Directive	Task	Example
.MODEL	Sets memory model	`.MODEL SMALL`
.STACK	Defines stack size	`.STACK 100H`
.DATA	Start data segment	`.DATA`
.CODE	Start code segment	`.CODE`
DB	Define Byte (8-bit)	`VAR DB 10H`
DW	Define Word (16-bit)	`VAR DW 1234H`
DD	Define Double word (32-bit)	`VAR DD 12345678H`
EQU	Define constant	`MAX EQU 100`
ORG	Set origin address	`ORG 100H` (for .COM files)
END	End of program	`END MAIN`

Pseudo Opcodes (Pseudo Instructions)

Instructions that look like opcodes but are handled by assembler.

Pseudo Opcode	Meaning	Example
ASSUME	Tells assembler about segment registers	`ASSUME CS:CODE, DS:DATA`
SEGMENT	Defines a segment	`DATA SEGMENT`
ENDS	End of segment	`DATA ENDS`
PUBLIC	Makes symbol available to other modules	`PUBLIC MY_PROC`
EXTRN	Declares external symbol	`EXTRN PRINT:NEAR`

ELI5:
Directives = Recipe instructions to the chef (assembler)
Pseudo opcodes = Special cooking terms that chef understands but aren't actual cooking actions

4(b) CF and OF after MUL BX and IMUL BX

Given: AX = FFFFH, BX = FFFFH

MUL BX (Unsigned Multiplication)

text

```
MUL BX  →  (DX:AX) = AX × BX
         = FFFFH × FFFFH
         = 65535 × 65535
         = 4,294,836,225
         = FFFE 0001H (32-bit result)

DX = FFFEH
AX = 0001H
```

Flags:

- **CF = 1** (because result > 16 bits, high word in DX ≠ 0)
- **OF = 1** (same reason)

IMUL BX (Signed Multiplication)

Interpret FFFFH as signed = **-1**

text

```
IMUL BX  →  (DX:AX) = (-1) × (-1) = +1
         = 0000 0001H

DX = 0000H
AX = 0001H
```

Flags:

- **CF = 0** (result fits in 16 bits)
- **OF = 0** (no overflow)

Summary Table

Instruction	AX	BX	DX:AX Result	CF	OF
MUL BX (unsigned)	FFFFH (65535)	FFFFH (65535)	FFFE 0001H	1	1
IMUL BX (signed)	FFFFH (-1)	FFFFH (-1)	0000 0001H	0	0

ELI5:

- **MUL** = "How many total?" (65535 × 65535 = huge number)
- **IMUL** = "How much debt?" (-$1 × -$1 = +$1 profit)

4(c) DOS and BIOS Interrupts

BIOS Interrupts (Basic Input/Output System)

Low-level hardware services stored in ROM.

INT	Function	Example
INT 10H	Video services	Set mode, write pixel, scroll
INT 13H	Disk services	Read/write sectors
INT 16H	Keyboard services	Read key, check status
INT 17H	Printer services	Send character to printer

DOS Interrupts (INT 21H)

Higher-level operating system services.

AH Value	Function	Example
01H	Read character from STDIN	`MOV AH, 1 INT 21H`
02H	Display character	`MOV DL, 'A' MOV AH, 2 INT 21H`
09H	Display string	`MOV DX, offset msg MOV AH, 9 INT 21H`
4CH	Exit program	`MOV AH, 4CH INT 21H`

Example: Using Both BIOS and DOS

assembly

```
; BIOS: Set video mode
MOV AH, 0           ; Set video mode function
MOV AL, 03H         ; 80x25 text mode
INT 10H             ; Call BIOS

; DOS: Display message
MOV AH, 9
MOV DX, OFFSET MSG
INT 21H

; DOS: Exit
MOV AH, 4CH
INT 21H
```

Differences

Aspect	BIOS	DOS
Location	ROM	RAM (loaded from disk)
Level	Low-level (hardware direct)	High-level (uses BIOS)

Aspect	BIOS	DOS
Speed	Faster	Slower
Portability	Hardware dependent	Hardware independent

ELI5:

- **BIOS** = Factory-installed remote (directly controls TV hardware)
- **DOS** = Cable box (sits between you and TV, adds features like recording)

Quick Revision Table – Section A

Term	ELI5
64-bit processor	8-lane highway (8 bytes at a time)
Interrupt	Phone ringing while reading
Multiplexing	Same road for cars & trucks at different times
ALE	Traffic light for address/data
BIU	Waiter bringing food
EU	Mouth chewing food
CS	Reading room (instructions)
DS	Book stacks (data)
Assembler directive	Recipe instruction to chef
MUL	Unsigned multiplication
IMUL	Signed multiplication
BIOS	Factory remote
DOS	Cable box

Good luck tomorrow! Key points to remember:

- Draw the 8086 architecture diagram
- Show address generation formula: Segment × 10H + Offset
- Explain CF vs OF with examples
- Remember MUL vs IMUL behavior with negative numbers

You've got this! 🚀

image.png

PNG 304.43KB

Here is the complete, exam-oriented solution for **Section B (Questions 5–8)** from your CSE3231 2019 paper.
I have included **assembly code, diagrams, tables, step-by-step explanations, real-world examples, and ELI5 summaries** as requested.

SECTION B

Question 5

5(a) Flag – Signed Overflow vs Unsigned Overflow

What is a Flag?

A **flag** is a single bit in the Flag Register that indicates the result of a previous arithmetic or logical operation.

Unsigned Overflow (Carry Flag – CF)

Occurs when: Result exceeds the maximum unsigned value (0 to 2^n-1)

Example (8-bit): 255 + 1 = 256

text

```
   1111 1111  (255)
+  0000 0001  (1)
= 1 0000 0000  (256) → CF = 1
```

Signed Overflow (Overflow Flag – OF)

Occurs when: Result exceeds signed range (-2^{n-1} to $+2^{n-1}$-1)

Example (8-bit): 127 + 1 = -128 (overflow!)

text

```
   0111 1111  (+127)
+  0000 0001  (+1)
=  1000 0000  (-128) → OF = 1
```

How Flags are Affected – Detection Rules

Operation	Unsigned (CF)	Signed (OF)
Addition	CF = 1 if carry out of MSB	OF = 1 if same sign inputs → opposite sign result
Subtraction	CF = 1 if borrow needed	OF = 1 if different sign inputs → opposite sign result
Multiplication	CF/OF = 1 if result > register size	Same
Division	Undefined if overflow	Undefined

Comparison Table

Aspect	Unsigned Overflow	Signed Overflow
Flag	CF (Carry)	OF (Overflow)
Range	0 to 2^n-1	-2^{n-1} to $+2^{n-1}$-1
8-bit range	0 to 255	-128 to +127
16-bit range	0 to 65535	-32768 to +32767
Example	255+1=256 → CF=1	127+1=-128 → OF=1

ELI5:

- **Unsigned (CF)** = Odometer (999 miles + 1 = 000, light comes on)
- **Signed (OF)** = Thermometer (+127°C + 1°C = -128°C → broken!)

5(b) Assembly Code to Reverse Bit Pattern of AX (Preserving Contents)

assembly

```
; Method 1: Using PUSH/POP to preserve AX, result in BX
REVERSE_AX1:
    PUSH AX             ; Save original AX
    MOV CX, 16          ; 16 bits to reverse
    XOR BX, BX          ; Clear BX for result
REV1:
    SHR AX, 1           ; Shift LSB into CF
    RCL BX, 1           ; Rotate CF into BX from left
    LOOP REV1
    ; BX now contains reversed bits
    POP AX              ; Restore original AX
    ; Result is in BX, AX unchanged
    RET

; Method 2: Using stack to preserve and return result in AX
REVERSE_AX2:
    PUSH AX             ; Save original
    MOV CX, 16
    XOR DX, DX          ; DX will hold result
    POP AX              ; Get original back into AX for
processing
    PUSH AX             ; Save again for later restore
REV2:
    SHR AX, 1
    RCL DX, 1
    LOOP REV2
    MOV AX, DX          ; AX = reversed
    POP BX              ; BX = original (discard or use)
    ; Result in AX, original lost (modify as needed)
    RET

; Method 3: Using XOR swap - result in AX, original lost
REVERSE_AX3:
    MOV CX, 8           ; 8 swaps for 16 bits
    MOV BX, AX
REV3:
    SHR BX, 1
    RCL AX, 1
    LOOP REV3
    RET
```

Example

text

```
AX before: 0x1234 = 0001 0010 0011 0100
After reverse:       0010 1100 0100 1000 = 0x2C48
```

ELI5:
Reversing bits = Mirror image. Like reading binary number backwards.
Method: Take one bit from right, put it into left of new register.

5(c) Program Segment Prefix (PSP)

Definition: A 256-byte (100H) block created by DOS before loading a program (COM or EXE).

PSP Structure

Offset	Size	Content
00H-01H	2 bytes	INT 20H instruction (exit program)
02H-03H	2 bytes	Segment of last memory block
05H-09H	5 bytes	FAR call to DOS
10H-11H	2 bytes	Environment segment
2CH-2DH	2 bytes	Environment segment pointer
80H	1 byte	Command line length
81H-FFH	127 bytes	Command line arguments (ASCIIZ)

Memory Layout Diagram

text

```
Address (Hex)    Content
_____________________________________________
0000            ┌──────────────────────────┐
                │  CD 20 (INT 20H)         │ ← PSP start
0002            │  Segment of last block   │
0004            │  Reserved                │
0005            │  FAR call to DOS (5 bytes) │
...             │  ...                     │
0080            │  Command line length (e.g., 05)
0081            │  'H' 'E' 'L' 'L' 'O'     │ ← arguments
...             │  ...                     │
00FF            │  End of PSP              │
0100            ├──────────────────────────┤
                │  Program starts here (.COM) │
                └──────────────────────────┘
```

Uses of PSP

Use	Description
Program exit	INT 20H at offset 0
Command line	Access arguments at 80H
Environment	Get environment variables
DOS dispatch	FAR call at offset 5

Example: Reading Command Line

assembly

```
; Command line is at PSP:80H
; Assume DS = PSP segment at program start
    MOV CL, [80H]        ; CL = command line length
    CMP CL, 0
    JE  NO_ARGS

    LEA SI, [81H]        ; SI points to arguments
    ; Process arguments...
```

ELI5:
PSP = "Welcome packet" DOS gives every program.
Contains: exit pass, command line arguments, environment settings.

Question 6

6(a) & 6(b) Range Problem of Conditional Jump Instructions

The Problem

Conditional jumps in 8086 are **short jumps** – they can only jump **-128 to +127 bytes** from the next instruction.

text

```
┌──────────────────────────────────────────────────┐
│  JNZ LABEL  (2 bytes)                            │
│      │                                           │
│      │  Can only jump within -128 to +127 bytes  │
│      ▼                                           │
│  LABEL: ...                                      │
└──────────────────────────────────────────────────┘
```

Why This is a Problem

Issue	Explanation
Limited range	Can't jump to far away labels
Large programs	Code may exceed 128 bytes between jump and target
Conditional logic	Complex conditions may need far jumps

Solutions

Solution 1: Reverse Condition + Far Jump

assembly

```
; Problem: Target is too far
        JNZ FAR_LABEL   ; ERROR - not allowed directly

; Solution:
        JZ NEAR_LABEL    ; Jump if ZERO (reverse condition)
        JMP FAR_LABEL    ; Unconditional far jump
NEAR_LABEL:
        ...              ; Rest of code
```

Solution 2: Use NEAR PTR (if within same segment)

assembly

```
        JNZ NEAR PTR FAR_LABEL   ; Near jump (up to 64KB)
```

Solution 3: Restructure Code

assembly

```
; Original (range problem)
        JNZ LABEL
        ... (more than 127 bytes)
LABEL:

; Restructured
        JZ  SKIP
        JMP LABEL
SKIP:
        ... (code that was between)
LABEL:
```

Diagram: Solution

text

```
Before (Problem):
┌──────────────────────────────────────────┐
│  JNZ TARGET    (2 bytes)                 │
│  ...                                      |
│  ... (130 bytes of code)                 │
│  TARGET: ...                             │
│  X (target too far, >127 bytes)          │
└──────────────────────────────────────────┘

After (Solution):
┌──────────────────────────────────────────┐
│  JZ  NEAR_LABEL   (2 bytes)              │
│  JMP TARGET       (3 bytes)              │
│  NEAR_LABEL:                             │
│  ... (code that was between)             │
│  TARGET: ...                             │
└──────────────────────────────────────────┘
```

ELI5:
Conditional jump = short step ladder (only reaches nearby)
Solution = If you can't reach, take one small step in opposite direction, then use a long ladder.

6(c) Masking – Set MSB and LSB of AL

Masking: Using AND/OR to modify specific bits while preserving others.

Problem: Set bit 7 (MSB) and bit 0 (LSB) of AL

AL bits: b7 b6 b5 b4 b3 b2 b1 b0

Solution:

assembly

```
OR  AL, 10000001B   ; Set bit7 and bit0 to 1
; OR with 1 sets bit, OR with 0 preserves
```

Explanation

Bit position	7	6	5	4	3	2	1	0
Original AL	x	x	x	x	x	x	x	x
Mask (OR)	1	0	0	0	0	0	0	1
Result	1	x	x	x	x	x	x	1

Other Masking Operations

Operation	Effect	Example
AND with 0	Clear bits	`AND AL, 11110000B` (clear lower 4 bits)
AND with 1	Preserve bits	`AND AL, 11111111B`
OR with 0	Preserve bits	`OR AL, 00000000B`
OR with 1	Set bits	`OR AL, 00001111B` (set lower 4 bits)
XOR with 1	Toggle bits	`XOR AL, 00001111B` (flip lower 4 bits)

ELI5:
Masking = Painting with stencil.
The mask `10000001B` is a stencil that only paints the first and last spots.

Question 7

7(a) Near Type vs Far Type Procedures

NEAR Procedure

Aspect	Description
Segment	Same segment as caller (CS same)
CALL size	3 bytes
RET size	1 byte
Offset	Only offset (16-bit) pushed on stack
Use	Small programs, same code segment

CALL NEAR: `CALL PROC_NAME` (3 bytes: opcode + offset)

FAR Procedure

Aspect	Description
Segment	Different segment from caller (CS changes)
CALL size	5 bytes
RET size	2 bytes
Offset	CS and offset (32-bit) pushed on stack
Use	Large programs, multiple code segments

CALL FAR: `CALL FAR PTR PROC_NAME` (5 bytes: opcode + offset + segment)

Comparison Table

Feature	NEAR	FAR
Segment change	No (same CS)	Yes (different CS)
Stack push	Only IP (2 bytes)	CS + IP (4 bytes)
CALL instruction size	3 bytes	5 bytes
RET instruction	`RET` (1 byte)	`RETF` (2 bytes)
Speed	Faster	Slower
Memory usage	Less	More

Feature	NEAR	FAR
Use case	Small programs	Large/modular programs

Diagram

text

```
NEAR CALL (Same Segment)
┌─────────────────────────────────────────────┐
│  Caller (CS: 1000H)                         │
│  ┌──────────────────┐                       │
│  │ CALL NEAR_PROC   │                       │
│  └──────────────────┘                       │
│           │                                 │
│           ▼                                 │
│  ┌──────────────────┐                       │
│  │ NEAR_PROC        │                       │
│  │ (CS: 1000H)      │                       │
│  └──────────────────┘                       │
└─────────────────────────────────────────────┘

FAR CALL (Different Segment)
┌─────────────────────────────────────────────┐
│  Caller (CS: 1000H)                         │
│  ┌──────────────────┐                       │
│  │ CALL FAR PROC    │                       │
│  └──────────────────┘                       │
│           │                                 │
│           ▼                                 │
│  ┌──────────────────┐                       │
│  │ FAR_PROC         │                       │
│  │ (CS: 2000H)      │                       │
│  └──────────────────┘                       │
└─────────────────────────────────────────────┘
```

ELI5:

- **NEAR** = Call a friend in the same building (just room number)
- **FAR** = Call a friend in a different building (need building number + room number)

7(b) What Happens When CALL and RET Execute

CALL Instruction Execution

Steps:

1. **Push IP** (offset of next instruction after CALL) onto stack
2. **Push CS** (current code segment) for FAR call only
3. **Load new CS:IP** from the called procedure's address
4. **Jump** to the procedure

Stack after NEAR CALL:

text

```
┌──────────────┐
│     IP       │ ← SP points here
├──────────────┤
│    ...       │
└──────────────┘
```

Stack after FAR CALL:

text

```
┌──────────────┐
│     CS       │ ← SP points here
├──────────────┤
│     IP       │
├──────────────┤
│    ...       │
└──────────────┘
```

RET Instruction Execution

Steps (NEAR RET):

1. **Pop IP** from stack (restore offset)

2. **Jump** back to calling location

Steps (FAR RET):

1. **Pop IP** from stack
2. **Pop CS** from stack
3. **Jump** back to calling location

Diagram

text

```
Before CALL:                    After CALL:
┌──────────────┐                ┌──────────────┐
│    Stack     │                │    Stack     │
├──────────────┤                ├──────────────┤
│              │                │   IP (ret)   │ ← SP
│              │                │   CS (ret)   │
└──────────────┘                └──────────────┘

     ↑                               ↑
    SP                              SP

After RET:                      Stack restored:
┌──────────────┐                ┌──────────────┐
│    Stack     │                │    Stack     │
├──────────────┤                ├──────────────┤
│              │                │              │ ← SP
│              │                │              │
└──────────────┘                └──────────────┘
```

ELI5:
CALL = Leave a sticky note (return address) on the stack, go to friend's house
RET = Read sticky note, come back home

7(c) Exchange 15th and 30th Elements in Word Array A

assembly

```
.DATA
    A DW 100 DUP(?)      ; Array of 100 words (16-bit each)

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Calculate offsets (each element = 2 bytes)
    ; 15th element index = 14 (0-based) → offset = 14 * 2 = 28
    ; 30th element index = 29 (0-based) → offset = 29 * 2 = 58

    LEA SI, A            ; SI = base address of array

    ; Method 1: Using registers
    MOV AX, [SI + 28]    ; AX = 15th element
    MOV BX, [SI + 58]    ; BX = 30th element

    MOV [SI + 28], BX    ; Store 30th into 15th position
    MOV [SI + 58], AX    ; Store 15th into 30th position

    ; Method 2: Using XCHG (requires one register)
    LEA SI, A
    MOV AX, [SI + 28]
    XCHG AX, [SI + 58]   ; Swap AX with 30th element
    MOV [SI + 28], AX    ; Store swapped value

    ; Method 3: Using PUSH/POP
    LEA SI, A
    PUSH [SI + 28]       ; Save 15th on stack
    PUSH [SI + 58]       ; Save 30th on stack
    POP [SI + 28]        ; Restore 30th to 15th position
    POP [SI + 58]        ; Restore 15th to 30th position

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Diagram: Array Element Exchange

text

```
Before Exchange:
┌──────────────────────────────────────────────────┐
│ A[0] │ A[1] │ ... │ A[14] │ ... │ A[29] │ ...     │
└──────────────────────────────────────────────────┘
                        ↑              ↑
                   15th element   30th element
                      (value X)      (value Y)

After Exchange:
┌──────────────────────────────────────────────────┐
│ A[0] │ A[1] │ ... │ A[14] │ ... │ A[29] │ ...     │
└──────────────────────────────────────────────────┘
                        ↑              ↑
                   15th element   30th element
                      (value Y)      (value X)
```

ELI5:
Swapping array elements = Two people swapping seats.
Need a temporary chair (register) to hold one person while the other moves.

Question 8

8(a) Addressing Modes – Based and Indexed

Addressing Mode: The method by which the CPU calculates the effective address of an operand.

Based Addressing Mode

Uses **BX** or **BP** as base pointer plus a displacement.

Syntax	Effective Address	Segment
`[BX + disp]`	DS:BX + disp	DS (default)
`[BP + disp]`	SS:BP + disp	SS (default)

Examples:

assembly

```
MOV AX, [BX + 10]     ; AX = memory at DS:BX+10
MOV [BP + 4], CX      ; Store CX at SS:BP+4
```

Indexed Addressing Mode

Uses **SI** or **DI** as index register plus a displacement.

Syntax	Effective Address	Segment
`[SI + disp]`	DS:SI + disp	DS
`[DI + disp]`	ES:DI + disp	ES (for strings)

Examples:

assembly

```
MOV AX, [SI + 20]     ; AX = memory at DS:SI+20
MOV [DI + 5], BL      ; Store BL at ES:DI+5
```

Based-Indexed Addressing

Combines base and index registers.

Syntax	Effective Address
`[BX + SI + disp]`	DS:BX + SI + disp
`[BP + DI + disp]`	SS:BP + DI + disp

Example:

assembly

```
MOV AX, [BX + SI + 8]     ; AX = memory at DS:BX+SI+8
```

Summary Table

Mode	Formula	Example	Use Case
Based	[BX + disp]	`MOV AX, [BX+10]`	Struct/record access
Indexed	[SI + disp]	`MOV AX, [SI+20]`	Array access

Mode	Formula	Example	Use Case
Based-Indexed	[BX + SI + disp]	`MOV AX, [BX+SI+4]`	2D arrays

Diagram: Based vs Indexed

text

```
Based Addressing (BX + 10)
┌──────────────────────────────────────────────────┐
│                        BX points to base          │
│                             │                     │
│                             ▼                     │
│  ┌──┬──┬──┬
```

```
.DATA
    STR1 DB 'Computer', 0
    STR2 DB 'Computer', 0
    MSG_EQUAL DB 'Strings are equal$'
    MSG_NOT DB 'Strings are not equal$'
.CODE
    LEA SI, STR1        ; DS:SI → first string
    LEA DI, STR2        ; ES:DI → second string
    MOV CX, 8           ; Compare 8 characters
    CLD                 ; Forward direction
    REPE CMPSB          ; Repeat while equal (ZF=1)

    JNE NOT_EQUAL       ; If ZF=0, strings differ

    ; Strings are equal
    LEA DX, MSG_EQUAL
    JMP DISPLAY

NOT_EQUAL:
    LEA DX, MSG_NOT

DISPLAY:
    MOV AH, 9
    INT 21H
```

String Instruction Prefixes

Prefix	Meaning	Condition
REP	Repeat CX times	Always
REPE / REPZ	Repeat while equal / zero	ZF = 1
REPNE / REPNZ	Repeat while not equal / not zero	ZF = 0

String Instructions Summary

Instruction	Operation	Example
MOVS	Move string	Copy block of memory
CMPS	Compare string	Check if two strings equal
SCAS	Scan string	Find character in string
LODS	Load string	Load from string into AL/AX
STOS	Store string	Store AL/AX into string

ELI5:

- **SCAS** = "Find Waldo" – scan string for character
- **CMPS** = "Spot the difference" – compare two strings
- **REP** = "Do this multiple times" – automatic loop

Quick Revision Table – Section B

Term	ELI5
Flag	Dashboard warning light
CF (Carry)	Odometer overflow light
OF (Overflow)	Thermometer broken light
Conditional jump range	Short step ladder (128 steps)
NEAR procedure	Friend in same building
FAR procedure	Friend in different building
Masking	Painting with stencil
Based addressing	"10 boxes after BX"
Indexed addressing	"20 boxes after SI"
SCAS	Find Waldo in string
CMPS	Spot difference between strings
PSP	Welcome packet from DOS

- Explain CF vs OF with numeric examples
- Show reverse bit code with RCL/RCR
- Draw PSP structure diagram
- Write actual string operation code

- Use based/indexed addressing in examples